

# **PRESENZ**

## **Software Engineering Project Report**

**Submitted by:**

**(<102316073>)DRISHTI**

**(<102316079>)SHREYA SHARMA**

**(<102316081>)TEJVIR SINGH GREWAL**

**(<102316085>)VISHESH KUMAR BANSAL**

**BE Third Year,CSE**

**Submitted to:**

**DR DAMINI ARORA**



**Computer Science and Engineering Department**

**TIET, Patiala**

# **INDEX**

| <b>S. No.</b> | <b>Assignment</b>                                | <b>Page No.</b> |
|---------------|--------------------------------------------------|-----------------|
| 1.            | Project Selection Phase                          | 4               |
| 1.1           | Software Bid                                     | 4               |
| 1.2           | Project Overview                                 | 7               |
| 2.            | Analysis Phase                                   | 14              |
| 2.1           | Use Cases                                        | 14              |
| 2.1.1         | Use Case Diagrams                                | 14              |
| 2.1.2         | Use Case Templates                               | 15              |
| 2.2           | Activity Diagram and Swimlane Diagrams           | 18              |
| 2.3           | Data Flow Diagrams (DFDs)                        | 20              |
| 2.3.1         | DFD Level 0                                      | 20              |
| 2.3.2         | DFD Level 1                                      | 20              |
| 2.3.3         | DFD Level 2                                      | 21              |
| 2.4           | Software Requirement Specification (IEEE Format) | 23              |
| 2.5           | User Stories and Story Cards                     | 36              |
| 3.            | Design Phase                                     | 48              |
| 3.1           | Class Diagram                                    | 48              |
| 3.2           | Sequence Diagram                                 | 49              |
| 3.3           | Collaboration Diagram                            | 51              |
| 3.4           | State Chart Diagrams                             | 52              |
| 4.            | Implementation                                   | 55              |
| 4.1           | Component Diagrams                               | 55              |

| <b>S. No.</b> | <b>Assignment</b>     | <b>Page No.</b> |
|---------------|-----------------------|-----------------|
| 4.2           | Deployment Diagrams   | 55              |
| 4.3           | Screenshots           | 56              |
| 5.            | Testing               | 58              |
| 5.1           | Test Plan             | 58              |
| 5.2           | Test Cases            | 60              |
| 5.3           | Test Reports by Peers | 63              |

# **1. Project Selection Phase**

## **1.1 Software Bid**

### **Software Bid/ Project Teams**

#### **UCS 503- Software Engineering Lab**

Group : 5

Dated: ..... 2025

**Team Name:**

**Team ID (will be assigned by Instructor):**

Please enter the names of your Preferred Team Members. :

- You are required to form a **three to four person** teams
- Choose your team members wisely. You will not be allowed to change teams.

| Name                 | Roll No   | Project Experience                                                                                                            | Programming Language used | Signature |
|----------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------|-----------|
| Vishesh Kumar Bansal | 102316085 | AI Student Attention Monitoring System, Social Media Management System , AI based dynamic traffic light management system.    | Python, JS,PL/SQL         |           |
| Tejvir Singh Grewal  | 102316081 | AI Student Attention Monitoring System, AI Smart Retail inventory Management System, Social Media Management System           | Python, JS, SQL, PLSQL    |           |
| Drishti              | 102316073 | Social Media Management System, AI Symptom Checker & Medicine Reminder System, Travel Management System                       | JS, Python, SQL           |           |
| Shreya Sharma        | 102316079 | Social Media Management, Movie Ticket Booking System, AI Symptom Checker & Medicine Reminder System, Travel Management System | JS, Python, SQL           |           |

### **Programming Language / Environment Experience**

List the languages you are most comfortable developing in, **as a team**, in your order of preference. Many of the projects involve Java or C/C++ programming.

1. JavaScript
2. Python
3. C/C++

### Choices of Projects:

Please select **4 projects** your team would like to work on, by order of preference: *[Write at-least one paragraph for each choice (motivation, reason for choice, feasibility analysis, etc.)]*

|               | Project Name                                                       | Unique Selling Point                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| First Choice  | <b>AI-Powered Personal Productivity Enhancer</b>                   | This project uses <b>behavioral analytics</b> to detect focus loss or fatigue based on computer interaction patterns, such as mouse movement, typing rhythm, and window-switching behavior, and then suggests smart breaks or focus reminders. Its unique value lies in being <b>non-intrusive and entirely local</b> , meaning no sensitive data is sent to the cloud. By integrating lightweight machine learning models in Python with tools like PyAutoGUI, pandas, and sklearn, it offers users a seamless, privacy-first way to understand and improve their digital work habits.                                                    |
| Second Choice | <b>Presenz</b>                                                     | <b>Presenz delivers reliable, proxy-proof attendance by combining real-time face recognition with GPS location verification. Its unique selling point lies in its dual authentication approach — not only does it confirm the user’s identity through AI-powered facial recognition, but it also verifies their exact location to ensure they are physically at the designated site. Built with Python (leveraging OpenCV, geopy, and advanced face recognition libraries), Presenz brings accuracy, transparency, and trust to the process, addressing the rising issue of attendance fraud in schools, universities, and workplaces.</b> |
| Third Choice  | <b>MediBox – Pill Reminder with Expiry &amp; Refill Detection-</b> | MediBox automates <b>medicine management</b> by using OCR to detect expiry dates and pill names directly from packaging images, removing the need for manual input. Its standout feature is the ability to <b>proactively remind users</b> about medication schedules, upcoming expiries, and refill needs—all without requiring external sensors or wearables. Implemented in Python with OCR and scheduling logic, it offers high practical value, especially for the elderly or patients with chronic illnesses.                                                                                                                        |
| Fourth Choice | <b>FacePointer</b>                                                 | FacePointer enables <b>hands-free computer control</b> by translating facial movements into cursor navigation, providing an inclusive tech solution for people with motor impairments. The USP lies in its <b>lightweight, hardware-free setup</b> , leveraging MediaPipe for real-time facial tracking and PyAutoGUI for smooth cursor control. This approach makes accessibility tools more approachable, affordable, and easy to deploy, all while                                                                                                                                                                                      |

|  |  |                                                                     |
|--|--|---------------------------------------------------------------------|
|  |  | delivering a meaningful impact within a short development timeline. |
|--|--|---------------------------------------------------------------------|

### **Additional Remarks/ Inputs**

Please tell us about any other factors that we should take into consideration (e.g., if you really would like to work on a project for some particularly convincing reason).

.....

## 1.2 project overview

### Executive Summary

Marking attendance is something we do almost every day — in schools, colleges, offices, or even at events. Yet, the way it's usually done is slow, outdated, and often inaccurate. Calling out names, signing sheets, or swiping ID cards takes time and is open to misuse, like having a friend mark your presence when you're not actually there.

Our project aims to fix this with a smart, mobile-first attendance system that checks two things before marking you present:

1. Your face — so we know it's really you.
2. Your location — so we know you're actually where you're supposed to be.

This dual verification approach makes proxy attendance almost impossible and speeds up the process to just a few seconds. Everything is stored in the cloud instantly, so admins or teachers can see attendance in real time, from anywhere.

### Present a Solution

We're building a mobile app that's simple to use:

1. Open the app, log in, and tap "Mark Attendance".
2. The camera will take your picture and verify it against your stored profile.
3. At the same time, the app checks your GPS location to ensure you're in the right place.
4. If both checks pass, your attendance is recorded instantly in the cloud.

On the admin side, there's a dashboard to view attendance live, track reports, and even check the location from which each person marked their presence.

### Novelty / Unique Selling Point

1. **Two-step security:** Face + GPS check for ultimate authenticity.
2. **No expensive hardware:** Works on any smartphone.
3. **Instant updates:** Attendance shows up in real-time for admins.
4. **Scales easily:** Suitable for classrooms, offices, or events without changes in infrastructure.

## Objectives

1. Make attendance marking quick and effortless.
2. Remove the possibility of fake or proxy attendance.
3. Keep all records digital, secure, and easy to access.
4. Allow admins to track attendance live from anywhere.
5. Build a system that works for small teams and large organizations alike.

## Methodology

We will follow a structured software engineering methodology that combines requirement gathering, design, development, testing, deployment, and maintenance.

### Step 1: Requirement Analysis

1. Engage with teachers, managers, and students/employees to understand pain points of existing attendance systems.
2. Document functional requirements (face recognition, GPS, reports) and non-functional requirements (security, scalability, real-time performance).

### Step 2: System Design

1. UI/UX Design: Create mobile app wireframes including login, attendance capture, and reports.
2. Database Design: Define entities like Users, Attendance Records, and Admins.
3. Data Flow:
  - User captures face image
  - GPS data fetched
  - Request sent to backend
  - Verification performed
  - Attendance stored and visible in dashboard

### Step 3: Technology Selection

1. Backend & Cloud: Firebase (Firestore + Storage + Auth) for simplicity, scalability, and real-time sync.
2. Face Recognition: MediaPipe (lightweight, mobile-friendly).
3. Location Services: Native Android/iOS GPS APIs.
4. App Development: Flutter (cross-platform, Android & iOS).

### Step 4: Development

1. Phase 1: Build core User Interface with login, attendance button, and dashboard access.
2. Phase 2: Integrate face recognition module for user verification.
3. Phase 3: Add GPS-based geofencing to restrict attendance to specific locations.
4. Phase 4: Connect with Firebase backend for data storage and authentication.
5. Phase 5: Implement Admin Dashboard for monitoring.

#### Step 5: Testing

1. Unit Testing: Test each module (login, face detection, GPS fetch).
2. Integration Testing: Verify combined workflow (face + location + database).
3. Pilot Testing: Small group of users (e.g., 10 students in a class) to validate real-world conditions.
4. Performance Testing: Ensure the system can handle 250+ requests in 2 minutes without delays.

#### Step 6: Deployment

1. Deploy mobile app on Play Store (Android) and optionally App Store (iOS).
2. Configure Firebase cloud services and set up admin accounts.
3. Train teachers/managers on how to use the admin dashboard.

#### Step 7: Maintenance & Improvement

1. Monitor app performance and server logs.
2. Collect feedback from users and admins.
3. Regularly update app features (e.g., adding notifications, analytics).
4. Scale backend if user base grows beyond initial projections.

### **Deliverables / Outcomes**

1. A fully working **mobile app** for marking attendance.
2. Face recognition and GPS location verification.
3. Cloud-based database with secure records.
4. Admin dashboard for live monitoring and report generation.
5. User guide and technical documentation.

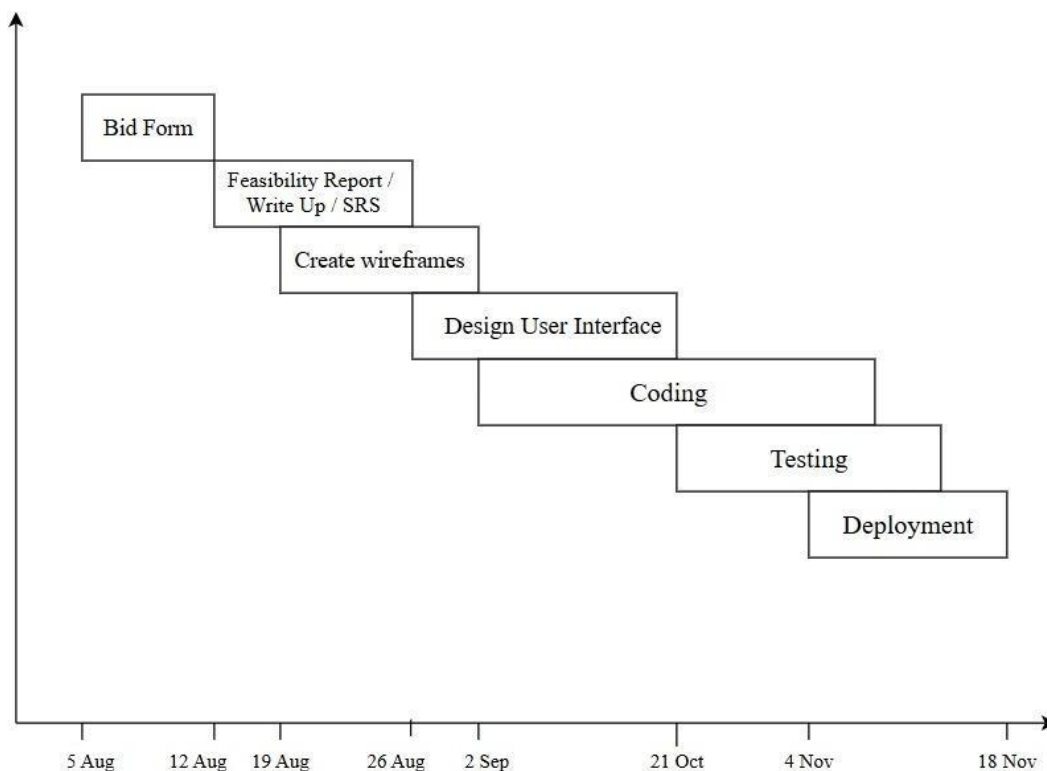
## Look & Feel of the Product

1. **For users:** Clean, modern interface with a single big button to mark attendance, camera preview, and instant confirmation.
2. **For admins:** Dashboard with a table of attendance records, location maps, and date filters.
3. **Design theme:** A calming blue-and-white gradient for trust and professionalism.
4. **Experience:** Fast, minimal steps, and zero confusion.

## Scope of Application

1. **Schools & Colleges** – Save classroom time and prevent proxies.
2. **Offices** – Track employee presence, integrate with HR systems.
3. **Events** – Manage guest check-ins efficiently.
4. **Field Teams** – Ensure workers are at the correct site before marking attendance.

## Timeline / Gantt Chart



## **Feasibility report**

Go / No Go Decision: Go

### **Aspects of Project Feasibility – Preparations and Analysis**

#### **A. Technical Feasibility**

The proposed solution — a mobile-based attendance system using face recognition and GPS verification — is technically practical and achievable with current technology. However, certain nuances need to be considered:

1. **Technology Availability:** Tools such as Firebase (for authentication, cloud storage, and real-time database), MediaPipe/OpenCV (for facial recognition), and native GPS APIs (for geolocation) are mature, widely used, and well-documented. This reduces the risk of implementation failure. However, a limitation is that face recognition performance may vary under different lighting conditions or device camera qualities, requiring optimization.
2. **Expertise:** The development team possesses prior experience in mobile app development, cloud integration, and AI-based recognition systems. Still, building a real-time, dual-authentication mechanism (face + GPS) demands careful testing and fine-tuning. Knowledge gaps, such as large-scale load testing or compliance with evolving data privacy laws, may require additional learning or consultation.
3. **Infrastructure Needs:** Since the system only requires standard smartphones, no special hardware costs are involved. However, cloud dependency introduces an element of reliance on internet connectivity and latency, which may affect user experience in areas with poor network coverage.
4. **Scalability:** Cloud-based auto-scaling ensures that the system can handle hundreds or even thousands of requests per minute. However, performance bottlenecks could emerge if face image processing is done entirely server-side without leveraging on-device computation. A hybrid approach (device-side face detection + server-side verification) may be considered to balance scalability with cost-efficiency.

#### **B. Managerial Feasibility**

1. **Team Competence:** The project team has the required skills to oversee development, deployment, and maintenance. The challenge will be ensuring effective coordination between frontend developers, backend engineers, and testers, especially since the project integrates multiple technologies (AI + GPS + cloud).

2. **Team Structure:** Roles are well-defined (Project Manager, Mobile Developers, Backend Engineer, UI/UX Designer, QA Tester). A risk is potential overlap in responsibilities, so clear task ownership will be critical.
3. **Processes:** The project will follow an Agile methodology with 2-week sprints. This ensures flexibility and quick iteration, though it may require disciplined sprint reviews and retrospectives to prevent scope creep.
4. **Experience:** While the team has prior experience in software delivery, scaling an application for real-time multiuser access might be relatively new, making careful risk monitoring necessary.

### **C. Economic Feasibility**

The app offers a cost-effective alternative to manual registers and biometric hardware systems.

1. **Cost Drivers:** Most development tools are open-source, reducing initial investment. The main expenses are cloud hosting (usage-based) and potential third-party API costs.
2. **ROI:** By reducing payroll fraud, proxy attendance, and wasted administrative hours, the system delivers strong operational efficiency. However, the economic benefit depends on adoption scale — larger institutions realize faster ROI than smaller ones.
3. **Breakeven Point:** For medium-sized organizations, the breakeven point is expected in 6–9 months. Smaller organizations may take longer, especially if cloud hosting costs outweigh manual tracking savings.

### **D. Financial Feasibility**

The project incurs minimal upfront costs, as development is conducted in-house and relies on free or low-cost tools. The primary financial considerations are cloud hosting charges and scaling costs. Since the app is also being built as part of a learning/research initiative, financial risks are negligible. However, long-term sustainability requires commitment to ongoing maintenance and technical support, which may indirectly incur human-resource costs.

### **E. Cultural Feasibility**

The app aligns well with the ongoing digital transformation in education and workplaces. Since most users are already comfortable with smartphones, adoption barriers are low. That said, resistance to change may occur in institutions accustomed to traditional methods. Training and awareness campaigns will be necessary to encourage smooth adoption.

### **F. Social Feasibility**

The system enhances punctuality, fairness, and transparency, ensuring equal accountability among all users. While this fosters trust, some users may express concerns about privacy (e.g.,

face data storage). Addressing these concerns through data encryption, anonymization, and transparent policies will be critical for social acceptance.

### **G. Safety Feasibility**

The app does not introduce any physical risks. Safety concerns primarily revolve around data security. End-to-end encryption, secure authentication, and GDPR/DPDP compliance will ensure that personal data remains protected. However, regular vulnerability assessments should be planned to maintain ongoing safety.

### **H. Political Feasibility**

The system complies with local labor, education, and data privacy regulations. Since privacy regulations vary across jurisdictions, the app must remain adaptable — e.g., allowing region-specific data storage policies. Potential political sensitivity may also arise if face recognition is perceived as surveillance, so emphasizing consent-based use is vital.

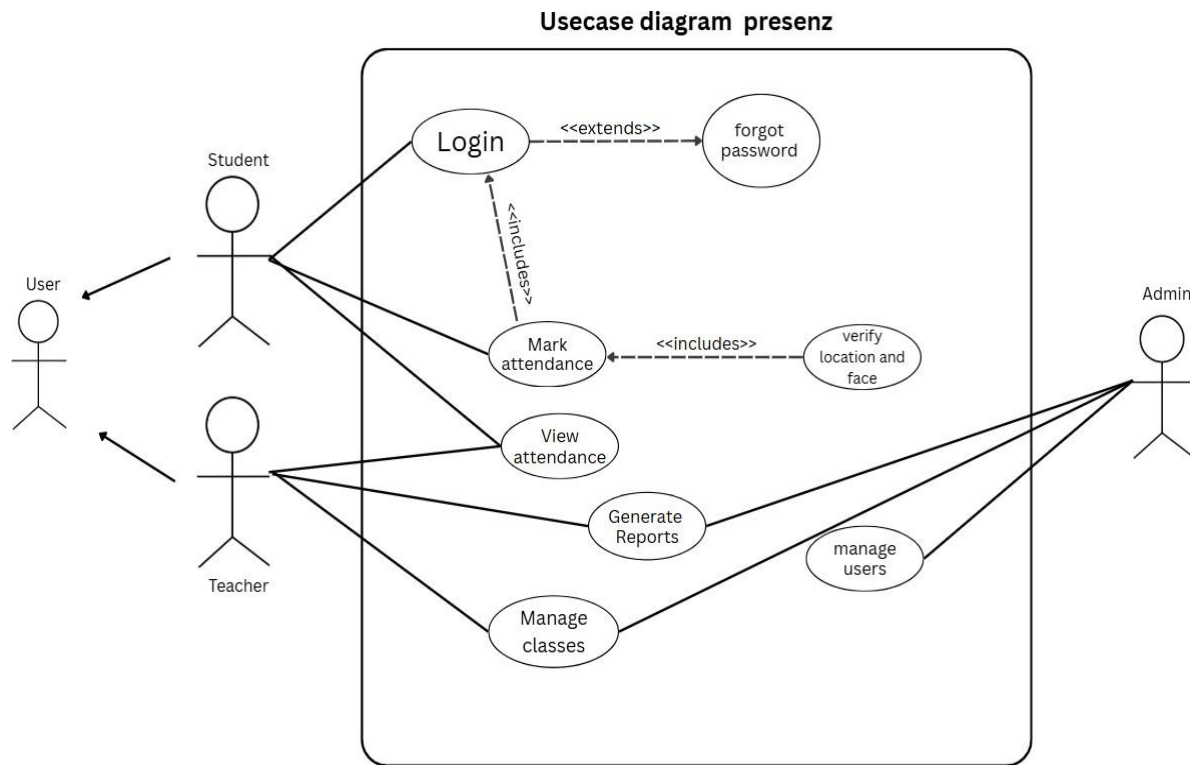
### **Technical Feasibility – Nuanced Analysis**

1. **Practicality of Technology :** The solution is technically practical since the core technologies are proven. Yet, real-world deployment will test robustness against challenges like poor lighting, device diversity, and network instability.
2. **Technology Availability:** All required tools are easily accessible, but reliance on third-party platforms (e.g., Firebase) creates a vendor lock-in risk, which should be acknowledged.
3. **Expertise and Schedule:** The team has the required expertise, and the proposed 8-week cycle is reasonable. However, unforeseen challenges — such as debugging GPS drift or optimizing face recognition — could extend timelines. Including a buffer period is advisable.
4. **Technology Maturity:** Face recognition and GPS APIs are sufficiently mature to support reliable deployment. Still, bias in AI models (e.g., reduced accuracy for different skin tones or facial features) remains an ongoing concern in the wider AI community. Mitigation strategies (like dataset balancing and fairness testing) should be integrated.

## 2. Analysis Phase

### 2.1 Use Cases

#### 2.1.1 Use-case Diagram



#### 2.1.2 Use-case templates

| Field                     | Details                                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-01                                                                                                                              |
| <b>Use Case Name</b>      | Login / Authentication                                                                                                             |
| <b>Actors</b>             | Student, Teacher, Admin                                                                                                            |
| <b>Description</b>        | Allows all users to securely log into the PRESENZ system using institutional credentials and assigned                              |
| <b>Preconditions</b>      | User must have valid credentials and an active account.                                                                            |
| <b>Postconditions</b>     | User gains access to their respective dashboard.                                                                                   |
| <b>Task Sequence</b>      | 1. User opens login page.<br>2. Enters credentials.<br>3. System verifies details.<br>4. Redirects to the corresponding dashboard. |
| <b>Alternate Flow</b>     | If credentials are invalid → system shows “Login Failed” and allows retry.                                                         |
| <b>Includes / Extends</b> | —                                                                                                                                  |
| <b>Exceptions</b>         | Account locked or server error → display error message.                                                                            |

| Field                     | Details                                                                                                                                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-02                                                                                                                                                                                                                                           |
| <b>Use Case Name</b>      | Mark Attendance                                                                                                                                                                                                                                 |
| <b>Actors</b>             | Student                                                                                                                                                                                                                                         |
| <b>Description</b>        | Enables students to mark their attendance using face recognition or QR code scanning.                                                                                                                                                           |
| <b>Preconditions</b>      | Student must be logged in camera and internet connection available.                                                                                                                                                                             |
| <b>Postconditions</b>     | Attendance is marked and stored in the system database.                                                                                                                                                                                         |
| <b>Task sequence</b>      | <ol style="list-style-type: none"> <li>1. Student selects "Mark Attendance."</li> <li>2. System prompts for face scan or QR scan.</li> <li>3. System verifies identity.</li> <li>4. Attendance recorded with timestamp and class ID.</li> </ol> |
| <b>Alternate Flow</b>     | If verification fails → retry prompt or manual teacher verification.                                                                                                                                                                            |
| <b>Includes / Extends</b> | <<include>> Face Recognition / QR Scan<br><<extends>> Verify Location via GPS<br><<includes>> Store Attendance Record                                                                                                                           |
| <b>Exceptions</b>         | Camera failure or network loss.                                                                                                                                                                                                                 |

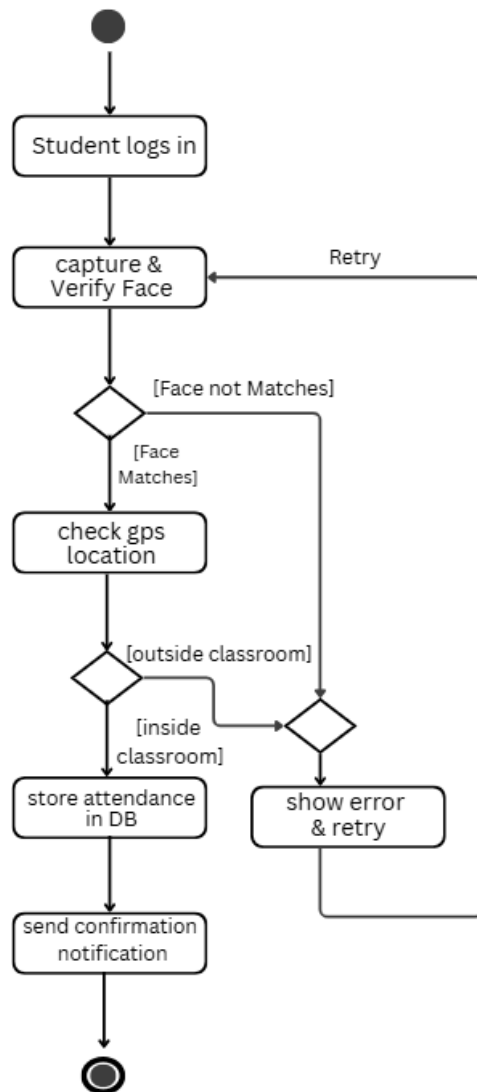
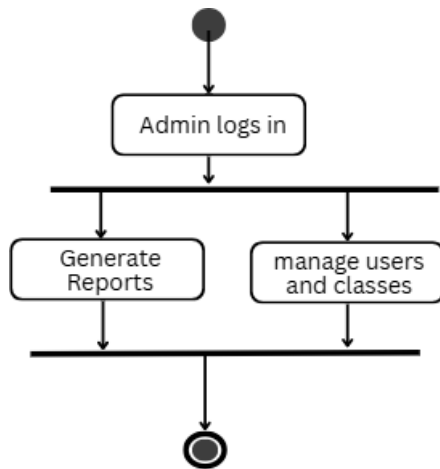
| Field                     | Details                                                                                                                                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-03                                                                                                                                                                                                                |
| <b>Use Case Name</b>      | Generate Reports                                                                                                                                                                                                     |
| <b>Actors</b>             | Teacher, Admin                                                                                                                                                                                                       |
| <b>Description</b>        | Allows generation of attendance reports for selected time ranges.                                                                                                                                                    |
| <b>Preconditions</b>      | Attendance records must exist in database.                                                                                                                                                                           |
| <b>Postconditions</b>     | Report generated and displayed/exported.                                                                                                                                                                             |
| <b>Task Sequence</b>      | <ol style="list-style-type: none"> <li>1. User selects "Generate Report."</li> <li>2. Chooses class/date range.</li> <li>3. System fetches data and creates report.</li> <li>4. Report displayed to user.</li> </ol> |
| <b>Alternate Flow</b>     | If no data found → show "No records available."                                                                                                                                                                      |
| <b>Includes / Extends</b> | <<extends>> Export Reports (PDF/CSV)<br><<extends>> Send Notifications                                                                                                                                               |
| <b>Exceptions</b>         | Database unavailable.                                                                                                                                                                                                |

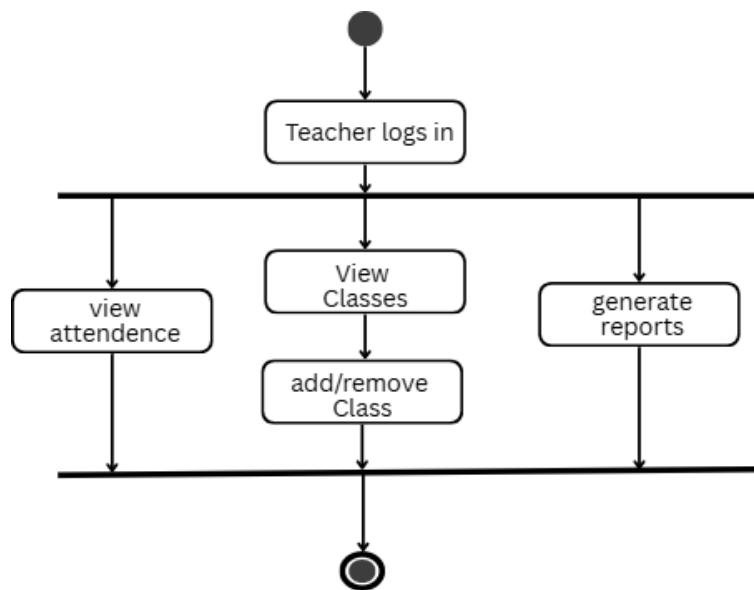
| Field                     | Details                                                                                                                                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-04                                                                                                                                                                                                       |
| <b>Use Case Name</b>      | Manage Users                                                                                                                                                                                                |
| <b>Actors</b>             | Admin                                                                                                                                                                                                       |
| <b>Description</b>        | Admin adds, edits, or removes student and teacher accounts.                                                                                                                                                 |
| <b>Preconditions</b>      | Admin is logged in and has permission.                                                                                                                                                                      |
| <b>Postconditions</b>     | User records updated in the database.                                                                                                                                                                       |
| <b>Task Sequence</b>      | <ol style="list-style-type: none"> <li>1. Admin selects "Manage Users."</li> <li>2. Adds/edits/deletes user data.</li> <li>3. System validates entries.</li> <li>4. System updates the database.</li> </ol> |
| <b>Alternate Flow</b>     | Duplicate or invalid data → show warning message.                                                                                                                                                           |
| <b>Includes / Extends</b> | <<includes>> Store / Update Data                                                                                                                                                                            |
| <b>Exceptions</b>         | Database connection error.                                                                                                                                                                                  |

| Field                     | Details                                                                                                                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-05                                                                                                                                                                                                            |
| <b>Use Case Name</b>      | Manage Classes                                                                                                                                                                                                   |
| <b>Actors</b>             | Admin                                                                                                                                                                                                            |
| <b>Description</b>        | Admin creates, assigns, or deletes class details.                                                                                                                                                                |
| <b>Preconditions</b>      | Admin logged in.                                                                                                                                                                                                 |
| <b>Postconditions</b>     | Updated class list stored in the database.                                                                                                                                                                       |
| <b>Task Sequence</b>      | <ol style="list-style-type: none"> <li>1. Admin opens "Manage Classes."</li> <li>2. Adds/edits/deletes class information.</li> <li>3. Assigns teachers to classes.</li> <li>4. System stores updates.</li> </ol> |
| <b>Alternate Flow</b>     | If teacher not found → prompt for re-selection.                                                                                                                                                                  |
| <b>Includes / Extends</b> | <<include>>Store / Update Data                                                                                                                                                                                   |
| <b>Exceptions</b>         | Data entry error or class conflict.                                                                                                                                                                              |

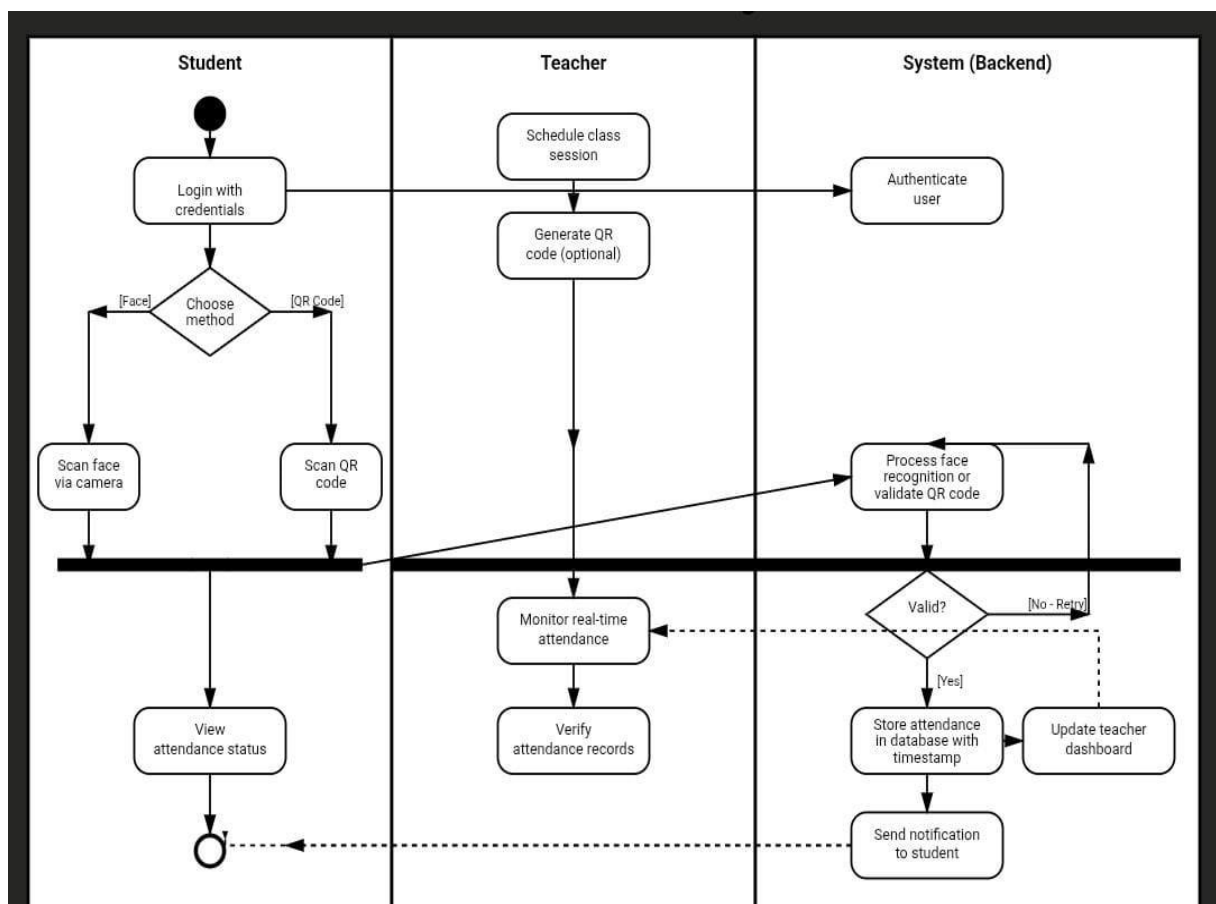
| Field                     | Details                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-06                                                                                                               |
| <b>Use Case Name</b>      | View Attendance Record                                                                                              |
| <b>Actors</b>             | Student, Teacher                                                                                                    |
| <b>Description</b>        | Enables users to view attendance history and performance trends.                                                    |
| <b>Preconditions</b>      | User logged in<br>attendance records exist.                                                                         |
| <b>Postconditions</b>     | Attendance data displayed.                                                                                          |
| <b>Task Sequence</b>      | 1. User opens "View Records."<br>2. System fetches relevant data.<br>3. Records displayed in table or chart format. |
| <b>Alternate Flow</b>     | No data available → show "No records."                                                                              |
| <b>Includes / Extends</b> | —                                                                                                                   |
| <b>Exceptions</b>         | Database timeout.                                                                                                   |

### 2.2.1 Activity Diagrams



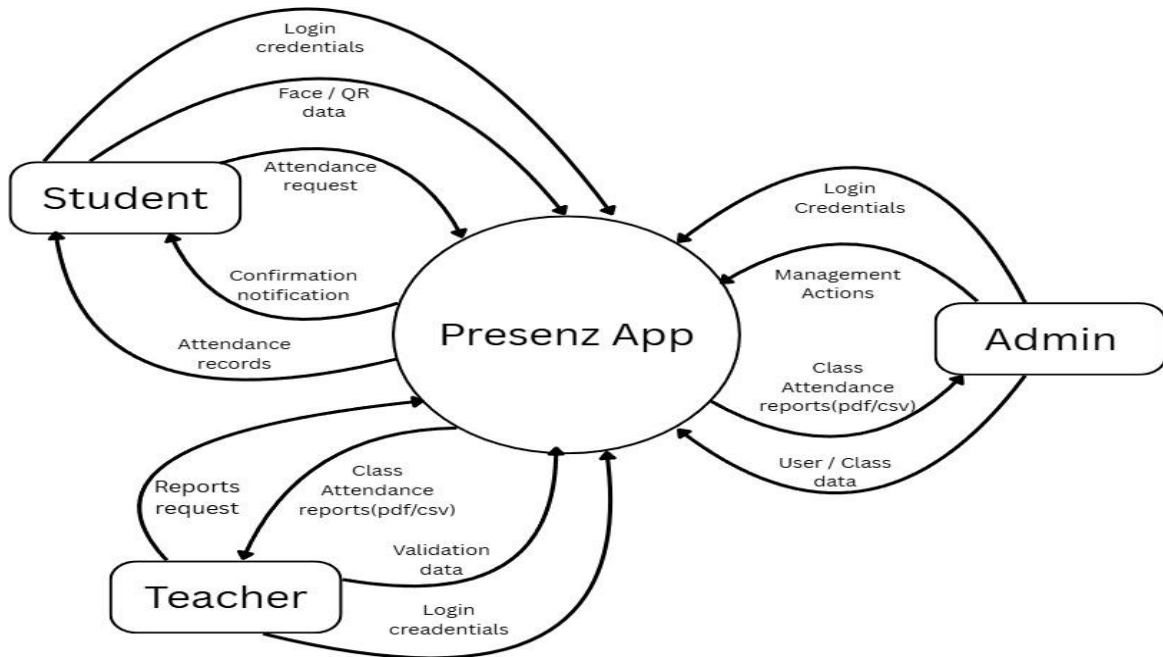


## 2.2.2 Swimlane Diagram

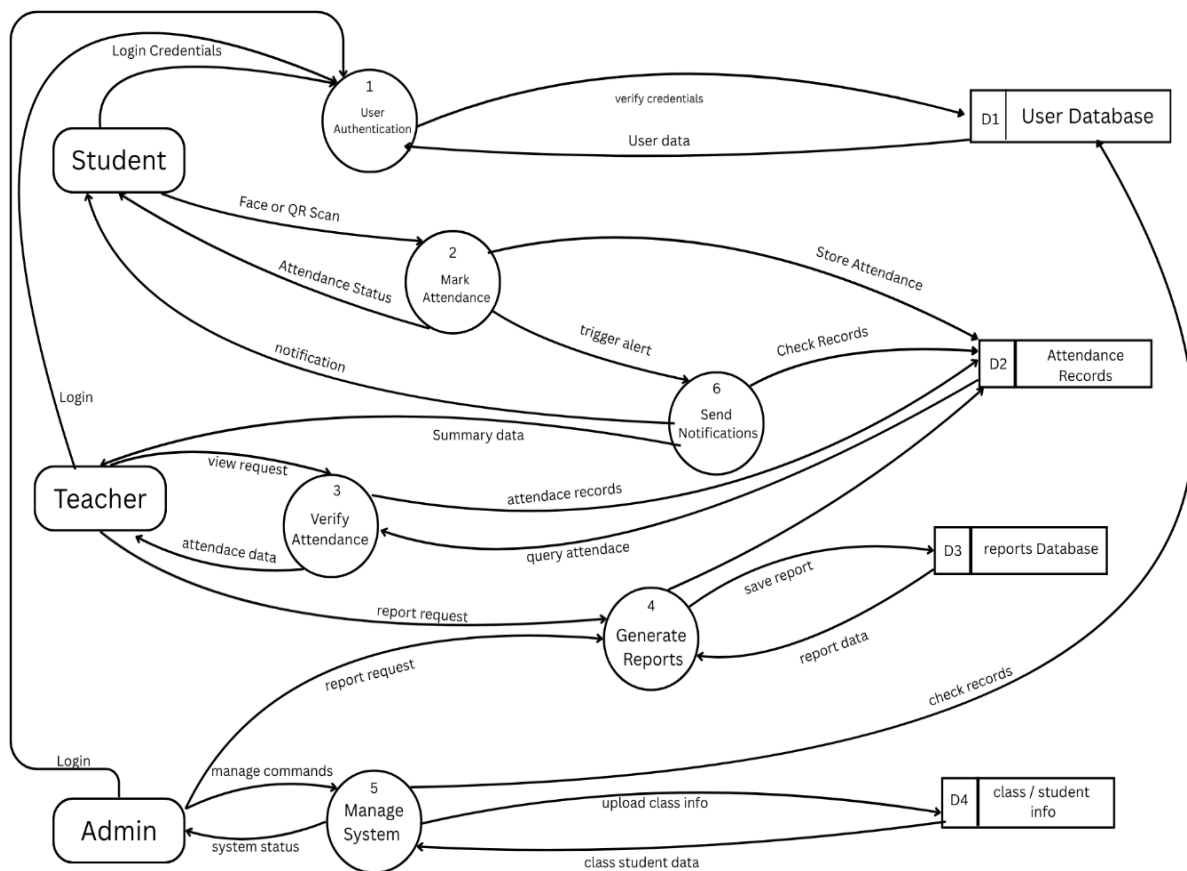


## 2.3 Data Flow Diagrams(DFDs)

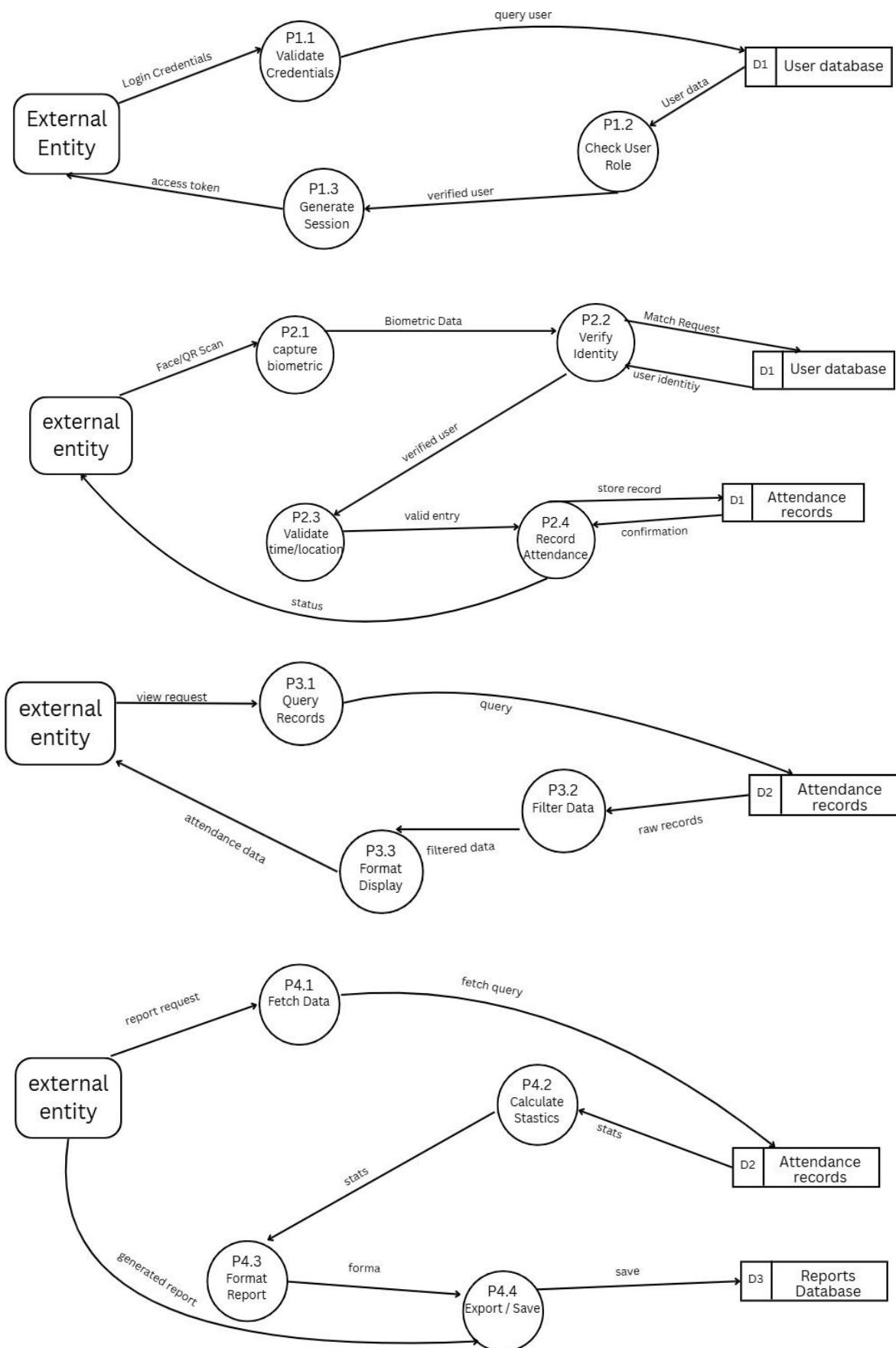
### 2.3.1 DFD Level 0

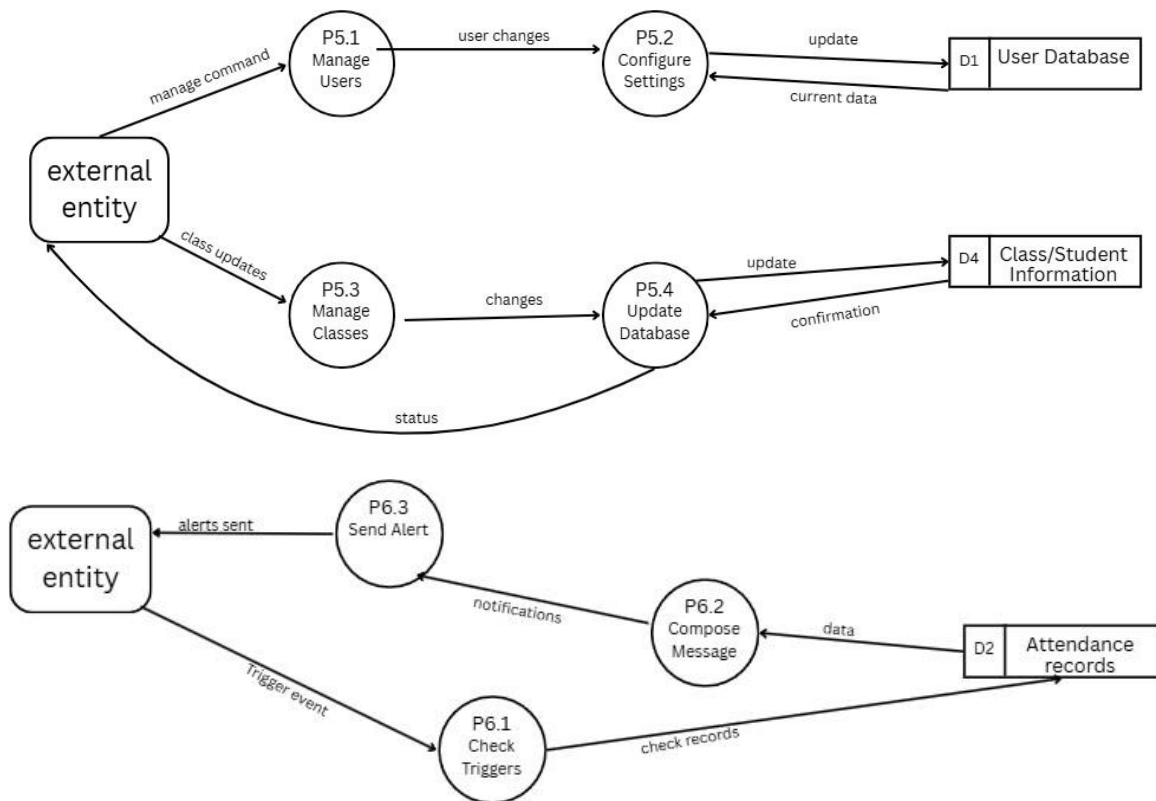


### 2.3.2 DFD Level 1



### 2.3.2 DFD Level 2





# **PRESENZ**

## **Software Requirements Specification (SRS) Report**

**Version: 1.0**

**Prepared by: Group 5**

**Date: 19 August 2025**



**Computer Science and Engineering Department**

**TIET, Patiala**

# Table of Contents

| Section                              | Subsections                                   |
|--------------------------------------|-----------------------------------------------|
| 1. Introduction                      | 1.1 Purpose                                   |
|                                      | 1.2 Document Conventions                      |
|                                      | 1.3 Intended Audience and Reading Suggestions |
|                                      | 1.4 Product Scope                             |
|                                      | 1.5 References                                |
| 2. Overall Description               | 2.1 Product Perspective                       |
|                                      | 2.2 Product Functions                         |
|                                      | 2.3 User Classes and Characteristics          |
|                                      | 2.4 Operating Environment                     |
|                                      | 2.5 Design and Implementation Constraints     |
|                                      | 2.6 User Documentation                        |
|                                      | 2.7 Assumptions and Dependencies              |
| 3. External Interface Requirements   | 3.1 User Interfaces                           |
|                                      | 3.2 Hardware Interfaces                       |
|                                      | 3.3 Software Interfaces                       |
|                                      | 3.4 Communications Interfaces                 |
| 4. System Features                   | 4.1 User Authentication                       |
|                                      | 4.1.1 Description and Priority                |
|                                      | 4.1.2 Stimulus/Response Sequences             |
|                                      | 4.1.3 Functional Requirements                 |
|                                      | 4.2 Attendance Marking                        |
|                                      | 4.2.1 Description and Priority                |
|                                      | 4.2.2 Stimulus/Response Sequences             |
|                                      | 4.2.3 Functional Requirements                 |
|                                      | 4.3 Attendance Database Storage               |
|                                      | 4.4 Report Generation                         |
|                                      | 4.5 Notifications                             |
|                                      | 4.6 Admin Dashboard                           |
| 5. Other Non-Functional Requirements | 5.1 Performance Requirements                  |
|                                      | 5.2 Security Requirements                     |
|                                      | 5.3 Scalability Requirements                  |
|                                      | 5.4 Usability Requirements                    |
| 6. Hardware & Software Requirements  | —                                             |
| 7. Constraints                       | —                                             |

# 1. Introduction

## 1.1 Purpose

The purpose of *Presenz* is to provide a modern, reliable, and automated solution for managing attendance in educational institutions. Traditional methods, such as manual roll calls and paper registers, are inefficient, time-consuming, and prone to human errors. Moreover, they leave room for unethical practices like proxy attendance, which undermines the credibility of attendance records.

This system solves these challenges by using a dual-verification process. First, it employs advanced face recognition technology to authenticate the identity of the student using the device's camera. Second, it uses location-based services powered by GPS and Google Maps geofencing to verify that the student is physically present within the classroom boundary. Attendance is marked only when both verifications are successful, thereby ensuring fairness, security, and integrity of the process.

By combining biometric verification with geolocation, *Presenz* significantly enhances classroom efficiency, saves valuable teaching time, and ensures that attendance data is stored securely in the cloud. Teachers and administrators gain real-time insights and analytics, while students benefit from a transparent and trustworthy system that records their presence accurately.

## 1.2 Document Conventions

This document has been prepared following standard software engineering practices. Functional requirements are described using clearly numbered points for precision and traceability. Non-functional requirements are organized into four major categories: performance, security, scalability, and usability. All system diagrams, such as use case models, entity-relationship diagrams, and system flows, are described textually for clarity, though they may also be represented visually in the design phase.

References to external technologies and services, such as Google Maps API, TensorFlow, and Firebase, are explicitly provided to establish their role in the system. This ensures that both technical and non-technical readers can easily follow the flow of the document.

### **1.3 Intended Audience and Reading Suggestions**

This document has been developed for different user groups, each with its own purpose:

1. Students, who should understand how attendance is recorded and how their presence is verified through face recognition and GPS validation. They will primarily engage with the mobile application.
2. Teachers, who will use the system to monitor attendance in real-time, validate classroom participation, and generate class-level summaries. Their role is more supervisory in nature.
3. Administrators, who will oversee the overall functioning of the system, manage student and teacher records, and generate institution-wide reports. They will mainly use the web-based dashboard.
4. Developers and testers, who will use this document as a foundation for system design, coding, and validation. They should refer to it for both functional requirements and technical constraints.

By addressing all of these groups, the document ensures clarity in communication and prevents ambiguity during development, deployment, and usage.

### **1.4 Product Scope**

*Presenz* is designed as a cross-platform system available on both mobile and desktop platforms. For students, it provides a simple and automated way to mark attendance by scanning their face, while the system validates their device's location using the Google Maps API. For teachers and administrators, it offers a reliable dashboard that ensures attendance records are up to date, error-free, and easily accessible.

The scope of the product can be summarized as follows:

1. Accuracy is ensured through the combination of biometric recognition and location verification, which prevents false or proxy attendance.
2. Time efficiency is achieved by eliminating the need for traditional roll calls, which consume valuable instructional time.
3. Transparency is provided by allowing students to view their attendance records anytime, thereby reducing disputes and confusion.

4. Analytics are made available for teachers and administrators, enabling them to generate attendance insights at daily, weekly, or monthly intervals.
5. Scalability is built into the system, making it applicable not just for schools and colleges but also for universities, professional training centers, and corporate organizations.

By offering these benefits, *Presenz* serves as a comprehensive solution that can adapt to a wide range of institutional needs.

## 1.5 References

The following references were used while preparing this document:

[1] K. E. Wiegers, *Software Requirements Specification: Recommended Practice*. IEEE, 1999. [Online].

Available: <https://ieeexplore.ieee.org/document/773959>

[2] Google Developers, *Google Maps Platform Documentation*. Google, 2025. [Online]. Available: <https://developers.google.com/maps>

[3] OpenCV.org, *OpenCV Documentation*. OpenCV, 2025. [Online]. Available: <https://docs.opencv.org>

[4] TensorFlow, *TensorFlow API Documentation*. Google, 2025. [Online]. Available: [https://www.tensorflow.org/api\\_docs](https://www.tensorflow.org/api_docs)

[5] Firebase, *Firebase Documentation*. Google, 2025. [Online]. Available: <https://firebase.google.com/docs>

[6] MongoDB, *MongoDB Atlas Documentation*. MongoDB, 2025. [Online]. Available: <https://www.mongodb.com/docs/atlas>

## **2. Overall Description**

### **2.1 Product Perspective**

*Presenz* is envisioned as a new-generation system to replace outdated manual and semi-automated methods of attendance tracking. While it can function independently, it also supports integration with existing institutional systems such as Learning Management Systems (LMS) and Enterprise Resource Planning (ERP) platforms through secure APIs. This allows institutions to incorporate *Presenz* into their digital infrastructure with minimal disruption.

The system is modular, meaning that individual components such as face recognition, location verification, reporting, and notifications can evolve independently without affecting the entire platform. This makes it adaptable to future upgrades, such as integrating with additional biometric authentication methods or supporting more advanced analytics.

### **2.2 Product Functions**

The system will support the following primary functions:

1. User authentication through secure, role-based login to differentiate between students, teachers, and administrators.
2. Attendance marking using a dual-check process: identity verification through face recognition and location verification through GPS geofencing.
3. Cloud-based storage of attendance records, which guarantees accessibility, reliability, and backup.
4. Report generation that provides daily, weekly, and monthly attendance summaries, with an option for export in multiple formats.
5. Notifications sent to students, teachers, and administrators to keep them informed of attendance status, summaries, or anomalies.
6. An administrative dashboard that allows management of users, courses, classes, and institution-level reports.

These functions ensure that the system is comprehensive, reliable, and aligned with the needs of all stakeholders.

## **2.3 User Classes and Characteristics**

The users of *Presenz* are categorized into three groups. Students will primarily interact with the mobile application to mark their attendance. Teachers will use the dashboard to verify attendance records, validate classroom participation, and address discrepancies. Administrators will oversee the institution-wide use of the system, ensuring proper configuration, managing users, and generating reports.

Each class of users requires a tailored interface that suits their role. Students need simplicity and speed, teachers require efficient monitoring tools, and administrators need powerful control and analytics. The design of *Presenz* ensures that these diverse needs are met without compromising usability.

## **2.4 Operating Environment**

The system is designed to operate across multiple platforms. On mobile devices, it will be developed using Flutter, ensuring compatibility with both Android and iOS. Teachers and administrators will access the system through a web dashboard, available on both Windows and Linux operating systems.

Stable internet connectivity is required at all times for synchronization, as attendance records are stored in the cloud. Devices must also be equipped with a functional camera for face recognition and GPS capabilities for location validation. Without these minimum requirements, the system cannot operate reliably.

## **2.5 Design and Implementation Constraints**

There are several constraints that may affect the system's performance. Face recognition accuracy may be reduced under poor lighting conditions or with low-quality device cameras. Location accuracy is dependent on GPS signals, which can sometimes be weak indoors; however, Wi-Fi-assisted location services may mitigate this issue.

The system also relies on continuous internet connectivity to synchronize data with the cloud, meaning offline functionality will be limited. Finally, data privacy and security standards, such as GDPR and IT regulations, must be followed to ensure compliance with global and national laws.

## **2.6 User Documentation**

The system will be supported by comprehensive documentation. Students, teachers, and administrators will have access to tailored user guides that explain their specific workflows. In addition, an FAQ and troubleshooting section will be integrated into the mobile application to resolve common issues quickly.

For administrators and technical staff, training manuals will be provided to assist with initial setup, configuration, and institutional management. These resources will ensure smooth adoption and reduce the learning curve associated with the system.

## **2.7 Assumptions and Dependencies**

The development of this system is based on certain assumptions. It is assumed that all students and teachers will have access to smartphones or laptops equipped with functioning cameras and GPS sensors. Reliable internet connectivity is assumed to be available during teaching hours.

The system is also dependent on external APIs such as Google Maps for geolocation and OpenCV/TensorFlow for face recognition. The continued availability and support of these services are crucial for the system's functionality.

# **3. External Interface Requirements**

## **3.1 User Interfaces**

The system will include different user interfaces depending on the role of the user. Students will first encounter a secure login screen where they authenticate themselves using institutional credentials. Once logged in, they will be directed to the attendance screen. On this screen, the device's camera will activate automatically to perform face recognition, while the system simultaneously validates their GPS location within the classroom geofence. Attendance will be marked only if both conditions are satisfied.

Teachers will access a dashboard where they can view attendance summaries in real-time. This interface will provide options to filter attendance by date, course, or student. Teachers will also have the ability to confirm or review anomalies, such as repeated absences. Administrators will have a more advanced dashboard with institution-level tools for managing user accounts, courses, and class

schedules. They will also be able to view analytics through graphs and charts and export records for reporting purposes.

### **3.2 Hardware Interfaces**

The hardware requirements for the system are minimal but essential. For students, the mobile device or laptop must have two core components: a functioning camera for capturing facial images and a GPS sensor for location verification. Teachers and administrators will typically use laptops or desktops, which may not require GPS but must be connected to the internet.

On the backend, the system will rely on cloud servers capable of handling multiple requests simultaneously. These servers must be scalable to accommodate the varying load that comes from different institutions, ranging from small classrooms to large universities with thousands of users.

### **3.3 Software Interfaces**

The system will consist of both frontend and backend components. The frontend will be built using Flutter to ensure that a single codebase can serve Android and iOS platforms efficiently. The backend will be developed using Firebase services such as Authentication and Firestore, or alternatively using FastAPI with MongoDB for institutions requiring greater customization.

Several external APIs will be integrated into the system. For face recognition, libraries such as OpenCV and TensorFlow will be used. For location verification, Google Maps API will be used to provide accurate GPS and geofencing functionality. The system will also support exporting attendance records in widely used formats, including CSV and PDF, to ensure compatibility with existing reporting workflows.

### **3.4 Communications Interfaces**

All communications between client devices and the server will occur over HTTPS, ensuring secure and encrypted data transmission. The system will rely on consistent internet connectivity, either through Wi-Fi or mobile data, for all users. Institutions must ensure that classrooms have sufficient internet access to support multiple students logging in simultaneously.

## **4. System Features**

### **4.1 User Authentication**

Authentication will be role-based, ensuring that students, teachers, and administrators log in with the appropriate permissions. The login process will support institutional credentials, while OAuth-based secure login options can also be enabled. Session management will be handled using secure tokens to prevent unauthorized access.

### **4.2 Attendance Marking**

The attendance marking process will follow a strict verification pipeline. First, the system will capture the student's facial image using the device camera and compare it with the stored facial embeddings. Second, the system will check the GPS location of the device and verify it against classroom coordinates defined through geofencing. Attendance will be marked only when both conditions are met, ensuring integrity. This dual-layer verification prevents proxy attendance and enhances fairness across the classroom.

### **4.3 Attendance Database Storage**

Every attendance record will contain the student's ID, class, timestamp, and location log. These records will be encrypted and stored in a cloud database. Administrators will have access to historical records for analysis, while students will only have access to their personal attendance history. Data retention policies will be followed to comply with institutional and legal requirements.

### **4.4 Report Generation**

The system will provide report generation capabilities at multiple levels. Daily reports will provide quick insights into class-level participation, weekly reports will highlight trends, and monthly reports will serve as official institutional records. Reports can be exported in CSV or PDF formats, making them easy to integrate with existing workflows. Teachers can use these reports for grading or participation tracking, while administrators can use them for policy-making.

## **4.5 Notifications**

The system will make extensive use of notifications to keep users informed. Students will receive confirmation messages when their attendance is successfully recorded, as well as alerts in case of absence. Teachers will be notified with daily summaries of attendance for their classes, and administrators will receive alerts about anomalies, such as large-scale absenteeism or system errors. Notifications will ensure proactive communication across all levels of the institution.

## **4.6 Admin Dashboard**

The administrative dashboard will serve as the control centre of the system. Administrators will be able to manage student and teacher accounts, assign roles, and configure classes. The dashboard will also include institution-level analytics that highlight attendance patterns and problem areas. With these insights, administrators can take corrective actions such as policy changes or targeted student engagement.

# **5. Other Non-Functional Requirements**

## **5.1 Performance Requirements**

The system must scale to handle at least 250 attendance requests within two minutes, which is sufficient for a large classroom. Location verification should complete within five seconds to ensure smooth operation. The backend must be designed to handle high traffic, particularly during peak times when multiple classes start simultaneously.

## **5.2 Security Requirements**

All sensitive data must be encrypted both in transit and at rest. Authentication will be token-based to prevent session hijacking. Biometric data will not be stored as raw images but as encrypted embeddings that cannot be reverse-engineered. Role-based access control will ensure that students, teachers, and administrators can only access data relevant to their responsibilities.

### **5.3 Scalability Requirements**

The system must be capable of supporting multiple institutions, each with thousands of users. Cloud deployment will allow horizontal scaling, where new servers can be added as the number of users grows. The architecture will also support integration with additional APIs in the future, such as fingerprint scanning or iris recognition, without requiring major changes.

### **5.4 Usability Requirements**

The interface must remain simple and intuitive, even for users with limited technical skills. Students should be able to mark attendance within seconds, without navigating complex menus. Teachers and administrators should be able to generate reports and view analytics with minimal training. The system will follow accessibility guidelines to ensure inclusivity for differently-abled users.

## **6. Hardware and Software Requirements**

The successful deployment of the system requires the following minimum requirements:

1. The frontend will be developed using Flutter to ensure cross-platform compatibility.
2. The backend will use Firebase (Authentication and Firestore) or FastAPI with MongoDB for scalability and flexibility.
3. User devices must have a functioning camera for face recognition and a GPS sensor for location verification.
4. Reliable internet connectivity is required at all times to synchronize attendance records with the cloud.
5. Administrative systems must have modern web browsers to access the dashboard seamlessly.

## 7. Constraints

The system is subject to several constraints that may affect its performance. The accuracy of face recognition depends on external conditions such as lighting, camera resolution, and device processing power. Similarly, GPS accuracy may decrease indoors, although this can be partially offset by Wi-Fi-assisted location services.

Another constraint is the requirement for stable internet connectivity. Without reliable access, attendance data cannot be synchronized with the cloud, potentially causing delays or inaccuracies. Furthermore, the system must comply with legal frameworks such as the General Data Protection Regulation (GDPR) and local IT laws to ensure privacy and security. Institutions deploying the system must therefore ensure that policies and practices align with these regulations.

## 2.5 User Stories and Story Cards

1.

#0001

### STUDENT LOGIN

As a **student**, I want to **log in**, so I can **access the attendance system securely**.

UI Mockup:

Username:

Password:

☐ Remember me

[Forgot password?](#)

Login

### #0001 - Student Login

#### Confirmation Scenarios:

##### 1. Success Path:

- Valid credentials – redirect to student dashboard
- "Remember me" checked – store session cookie
- "Remember me" not checked – temporary session only

##### 2. Failure - Display Messages:

- Invalid username – "User not found"
- Invalid password – "Incorrect password, please try again"
- Account locked – "Too many failed attempts, contact admin"
- Network error – "Connection failed, please try again"

2.

#0002

### MARK ATTENDANCE - FACE RECOGNITION

As a **student**, I want to **mark attendance using face recognition**, so I can **quickly record my presence without physical contact**.

UI Mockup:



Capture

Cancel

### #0002 - Mark Attendance - Face Recognition

#### Confirmation Scenarios:

##### 1. Success Path:

- Face detected and matched – mark attendance with timestamp
- Store attendance record with location data
- Send confirmation notification to student
- Update attendance percentage

##### 2. Failure - Display Messages:

- No face detected – "Please position your face in the frame"
- Multiple faces – "Only one person should be in frame"
- Face not matched – "Identity verification failed, try again"
- Poor lighting – "Insufficient lighting, move to brighter area"
- Camera access denied – "Please enable camera permissions"

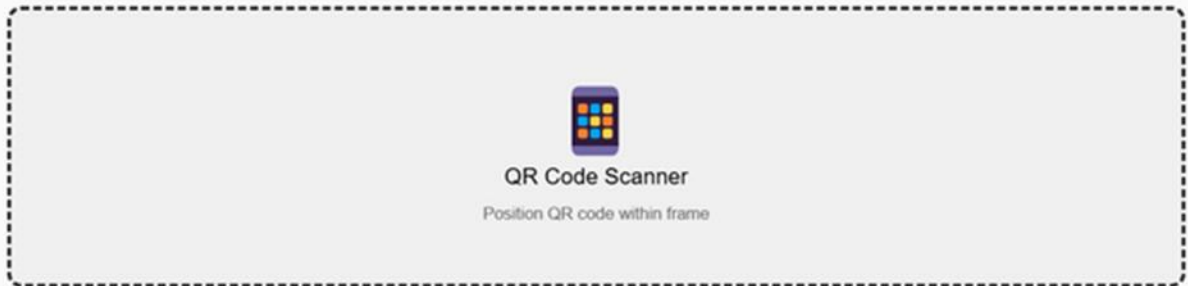
3.

#0003

### MARK ATTENDANCE - QR CODE

As a **student**, I want to **scan QR code to mark attendance**, so I can **have an alternative marking method**.

UI Mockup:



### #0003 - Mark Attendance - QR Code

#### Confirmation Scenarios:

##### 1. Success Path:

- Valid QR code – verify class and time
- Mark attendance successfully
- Display confirmation message
- Log attendance with timestamp and location

##### 2. Failure - Display Messages:

- Invalid QR code – "QR code not recognized"
- Expired QR code – "This session has ended"
- Wrong class – "QR code is for different class"
- Already marked – "Attendance already marked for this session"
- Outside time window – "Attendance window is closed"

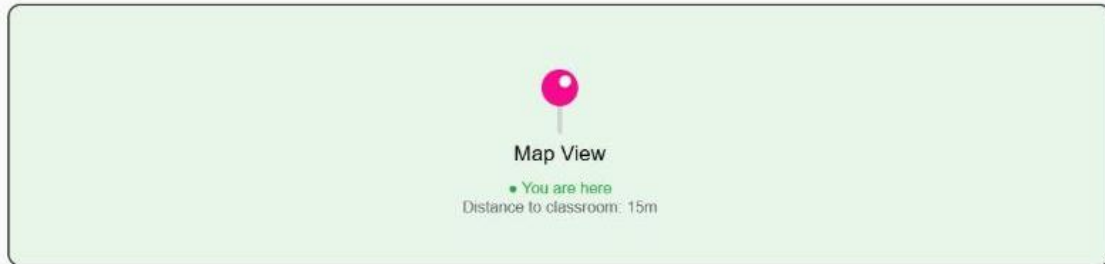
4.

#0004

### GPS LOCATION VERIFICATION

As a **student**, I want **my location to be verified automatically**, so I can **prove I am physically present in classroom**.

UI Mockup:



### #0004 - GPS Location Verification

#### Confirmation Scenarios:

##### 1. Success Path:

- Location within boundary – allow attendance marking
- Store GPS coordinates in location log
- Proceed to identity verification step

##### 2. Failure - Display Messages:

- Outside boundary – "You are not within classroom area"
- GPS disabled – "Please enable location services"
- Weak GPS signal – "Unable to determine location accurately"
- Location permission denied – "Location access required"

5.

#0005

### VIEW PERSONAL ATTENDANCE

As a **student**, I want to **view my attendance history**, so I can **track my attendance record**.

#### UI Mockup:

Filter:  Date:  

|                                        |
|----------------------------------------|
| ✓ Present - Oct 10, 2025 - Mathematics |
| X Absent - Oct 9, 2025 - Physics       |
| ✓ Present - Oct 8, 2025 - Chemistry    |

### #0005 - View Personal Attendance

#### Confirmation Scenarios:

##### 1. Success Path:

- Display all attendance records
- Show attendance percentage per subject
- Highlight low attendance warnings
- Filter by date range works correctly
- Export report successfully

##### 2. Failure - Display Messages:

- No records found – "No attendance data available"
- Export failed – "Unable to generate report"
- Invalid date range – "End date cannot be before start date"

6.

#0006

## MANAGE USERS

As an **admin**, I want to **add, edit, and remove users**, so I can **maintain accurate user records**.

### UI Mockup:

**+ Add User**

| Name     | Role    | Actions                                                                                                                                                                 |
|----------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| John Doe | Student |   |

#0006 - Manage Users

### Confirmation Scenarios:

#### 1. Success Path:

- Add new user – create account and send credentials
- Edit user – update information successfully
- Delete user – deactivate and archive data
- Bulk upload – import multiple users from CSV
- Display confirmation message for all actions

#### 2. Failure - Display Messages:

- Duplicate email – "Email already exists"
- Invalid data – "Please fill all required fields"
- Delete failed – "Cannot delete user with active sessions"
- CSV error – "Invalid file format or missing columns"

7.

#0007

## TEACHER LOGIN

As a **teacher**, I want to **log in with my credentials**, so I can **access teacher features securely**.

### UI Mockup:

Username:

Password:

☐ Remember me

[Forgot password?](#)

Login

### #0007 - Teacher Login

### Confirmation Scenarios:

#### 1. Success Path:

- Valid teacher credentials – redirect to teacher dashboard
- Display assigned classes
- Load today's attendance summary

#### 2. Failure - Display Messages:

- Invalid credentials – "Login failed, please try again"
- Not a teacher account – "Insufficient permissions"
- Account suspended – "Contact administrator"

8.

#0008

## VIEW CLASS ATTENDANCE

As a **teacher**, I want to **view attendance for my classes**, so I can **monitor student participation**.

### UI Mockup:

|        |                                     |       |                                         |                                                                                   |
|--------|-------------------------------------|-------|-----------------------------------------|-----------------------------------------------------------------------------------|
| Class: | <input type="text" value="CS-101"/> | Date: | <input type="text" value="dd-mm-yyyy"/> |  |
|--------|-------------------------------------|-------|-----------------------------------------|-----------------------------------------------------------------------------------|

| Student    | Status    |
|------------|-----------|
| John Doe   | ✓ Present |
| Jane Smith | ✗ Absent  |

#0008 - View Class Attendance

### Confirmation Scenarios:

#### 1. Success Path:

- Display complete attendance list for selected class
- Show present/absent/late counts
- Real-time updates during class session
- Export class attendance list
- Highlight students with patterns

#### 2. Failure - Display Messages:

- No data available – "No attendance records for selected date"
- Class not found – "Invalid class selection"

9.

#0009

## ADMIN LOGIN

As an **admin**, I want to **log in with admin credentials**, so I can **access system management features**.

### UI Mockup:

The UI mockup shows a login form with the following elements:

- A "Username:" label followed by a text input field with placeholder text "Enter username".
- A "Password:" label followed by a text input field with placeholder text "Enter password".
- A checkbox labeled "Remember me" and a link "Forgot password?".
- A blue "Login" button.

### #0009 - Admin Login

#### Confirmation Scenarios:

##### 1. Success Path:

- Valid admin credentials – redirect to admin dashboard
- Display system overview and statistics
- Access to all management features

##### 2. Failure - Display Messages:

- Invalid credentials – "Admin authentication failed"
- 2FA required – "Enter two-factor authentication code"
- Session expired – "Please login again"

10.

#0010

## GENERATE REPORTS

As a **teacher**, I want to **generate attendance reports**, so I can **analyze trends and share with administration**.

UI Mockup:

Report Type: 

Monthly Report

Date Range: 

dd-mm-yyyy

 to 

dd-mm-yyyy

Format: ☐ PDF ☐ CSV

Preview

Generate

### #0010 - Generate Reports

#### Confirmation Scenarios:

##### 1. Success Path:

- Generate report with selected parameters
- Include attendance statistics and charts
- Show individual student summaries
- Preview before final generation
- Download in selected format

##### 2. Failure - Display Messages:

- Invalid date range – "Please select valid date range"
- No data available – "No records found for selected criteria"
- Generation failed – "Unable to create report, try again"

11.

#0011

## MANAGE CLASSES

As an **admin**, I want to **create and manage classes**, so I can **organize the academic structure**.

UI Mockup:

Report Type:

Date Range:  to

Format: ☒ PDF ☐ CSV

Preview

Generate

#0011 - Manage Classes

### Confirmation Scenarios:

#### 1. Success Path:

- Create new class with details
- Assign teacher to class
- Enroll students successfully
- Set class schedule and location boundaries
- Generate unique class QR codes

#### 2. Failure - Display Messages:

- Duplicate class – "Class code already exists"
- Invalid schedule – "Time conflict with existing class"
- Teacher unavailable – "Teacher already assigned to another class"
- Location error – "Invalid GPS coordinates"

12.

#0012

## VIEW SYSTEM DASHBOARD

As an **admin**, I want to **view system dashboard**, so I can **monitor overall system performance**.

UI Mockup:

**1,234**  
Total Students

**87%**  
Today's Attendance

#0012 - View System Dashboard

### Confirmation Scenarios:

#### 1. Success Path:

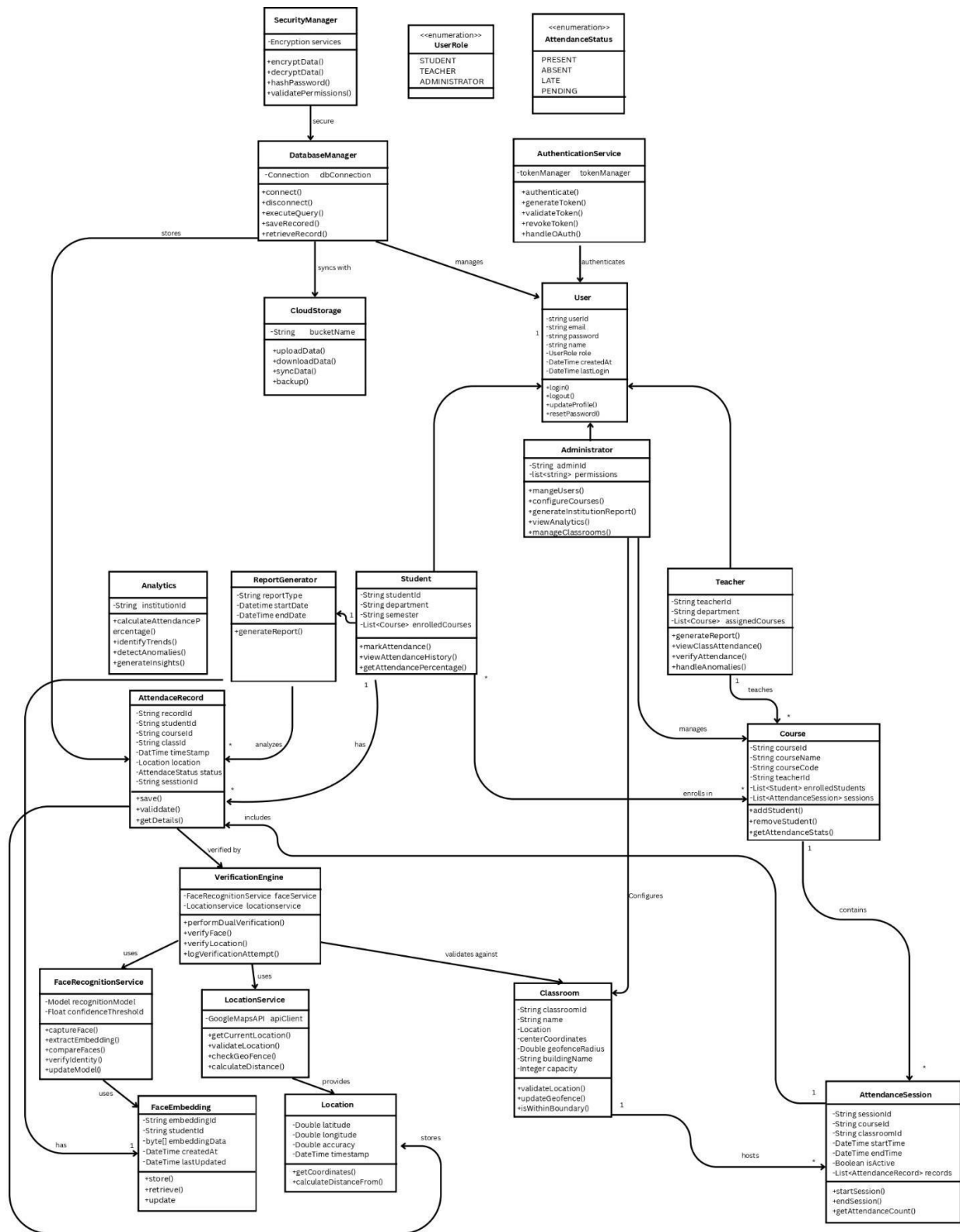
- Display total users (students, teachers, admins)
- Show today's attendance statistics
- Real-time system activity updates
- Alert notifications for issues
- Quick access to common admin tasks

#### 2. Failure - Display Messages:

- Data load error – "Unable to fetch statistics"
- Chart rendering failed – "Unable to display graphs"

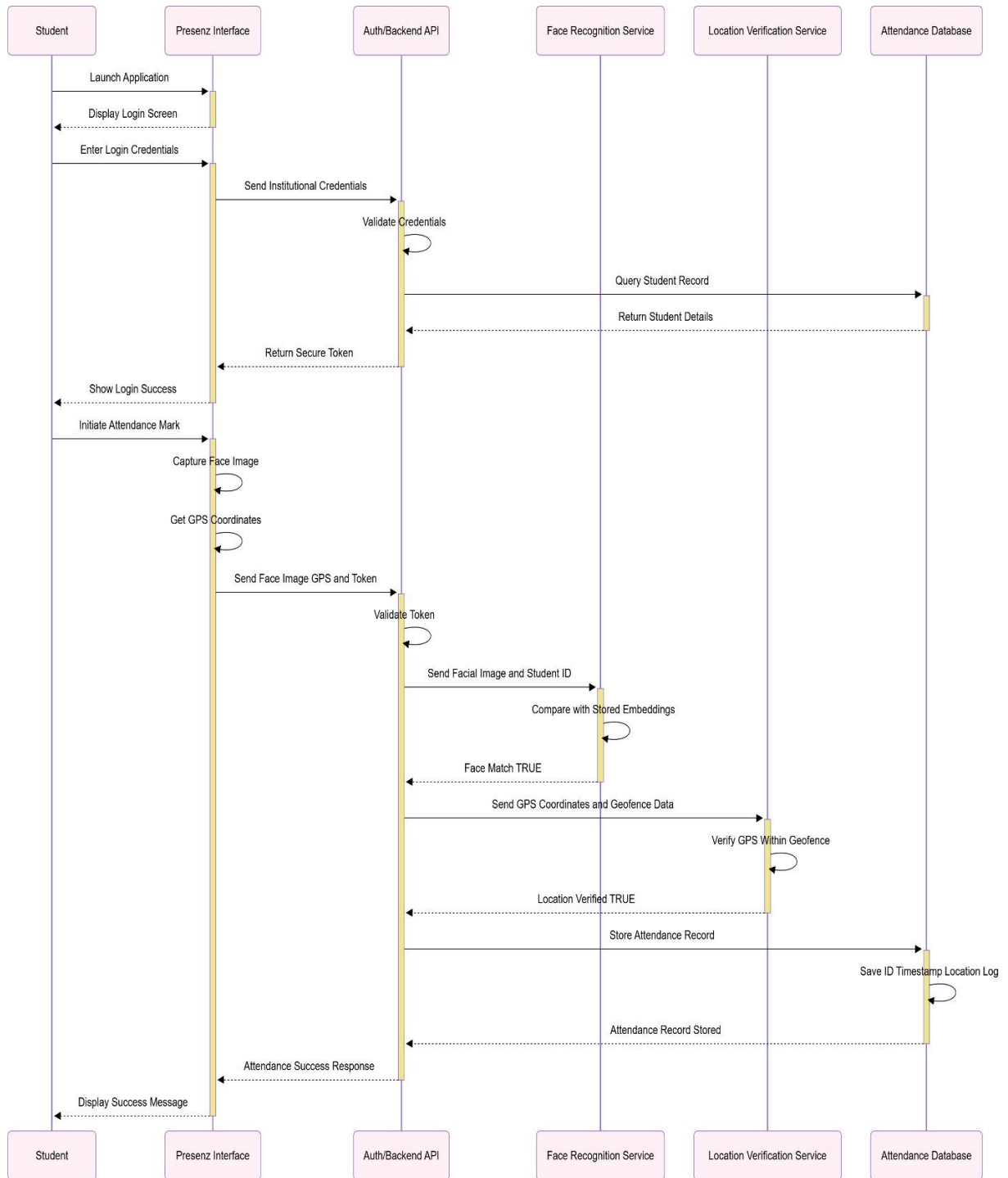
## 3. Design Phase

### 3.1 Class Diagram

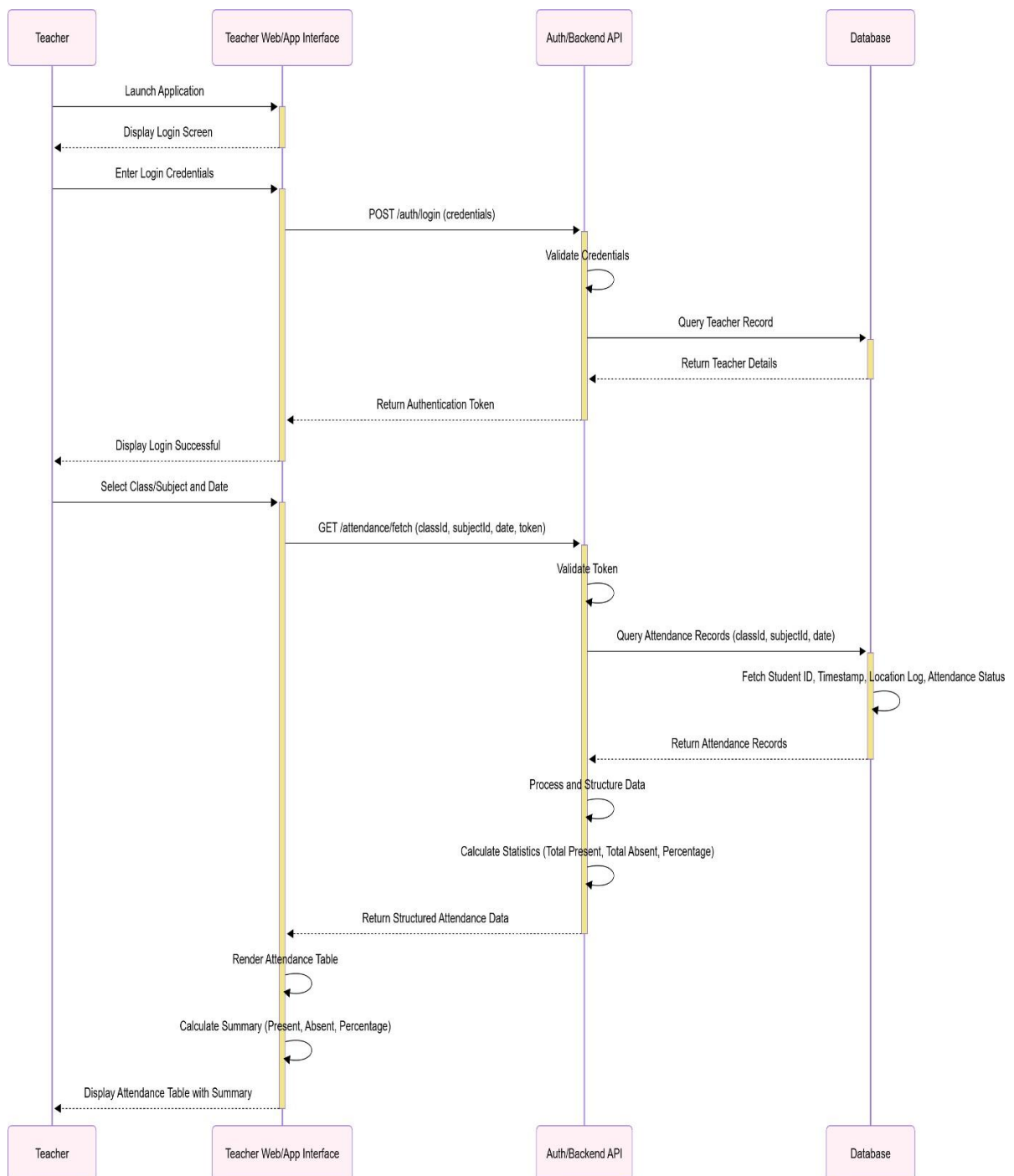


## 3.2 Sequence Diagram

### Case 1: Student Attendance Marking Sequence

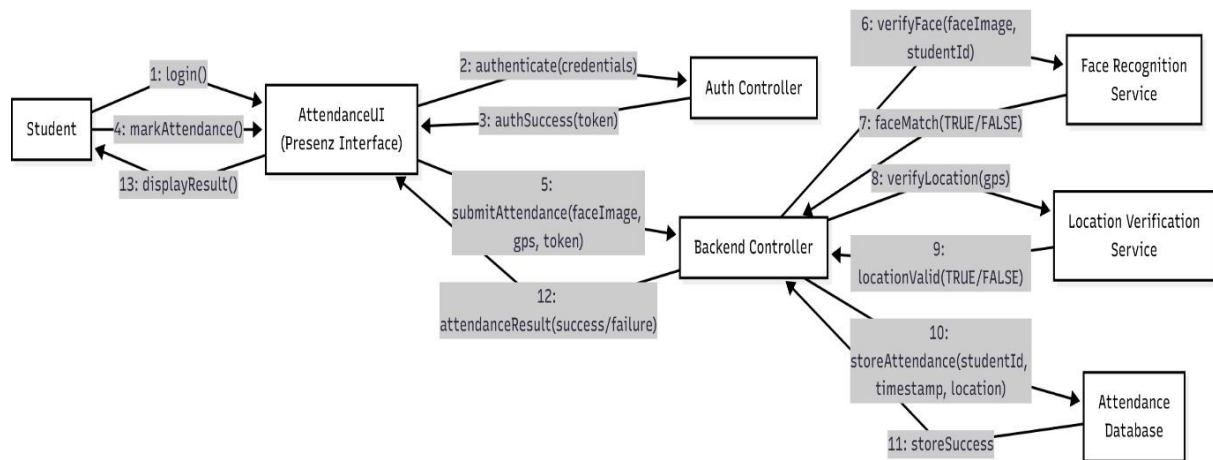


## Case 2: Teacher Attendance Viewing Sequence

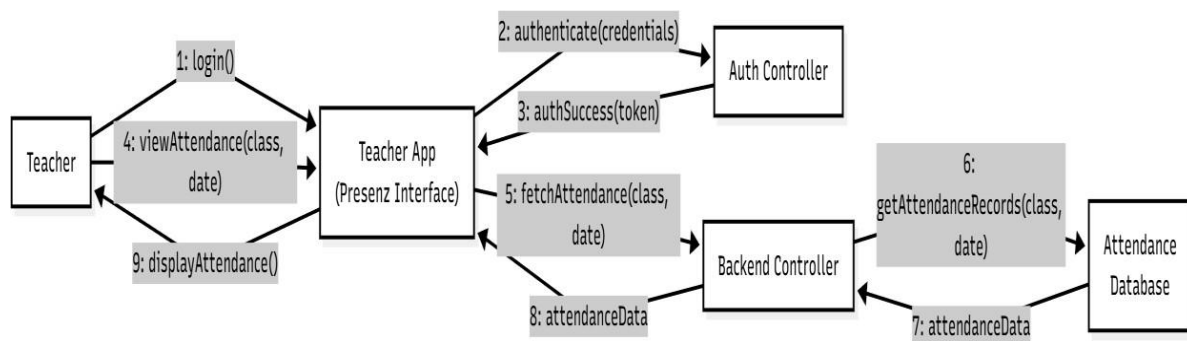


### 3.3 Collaboration Diagram

#### Case 1: Student Marks Attendance

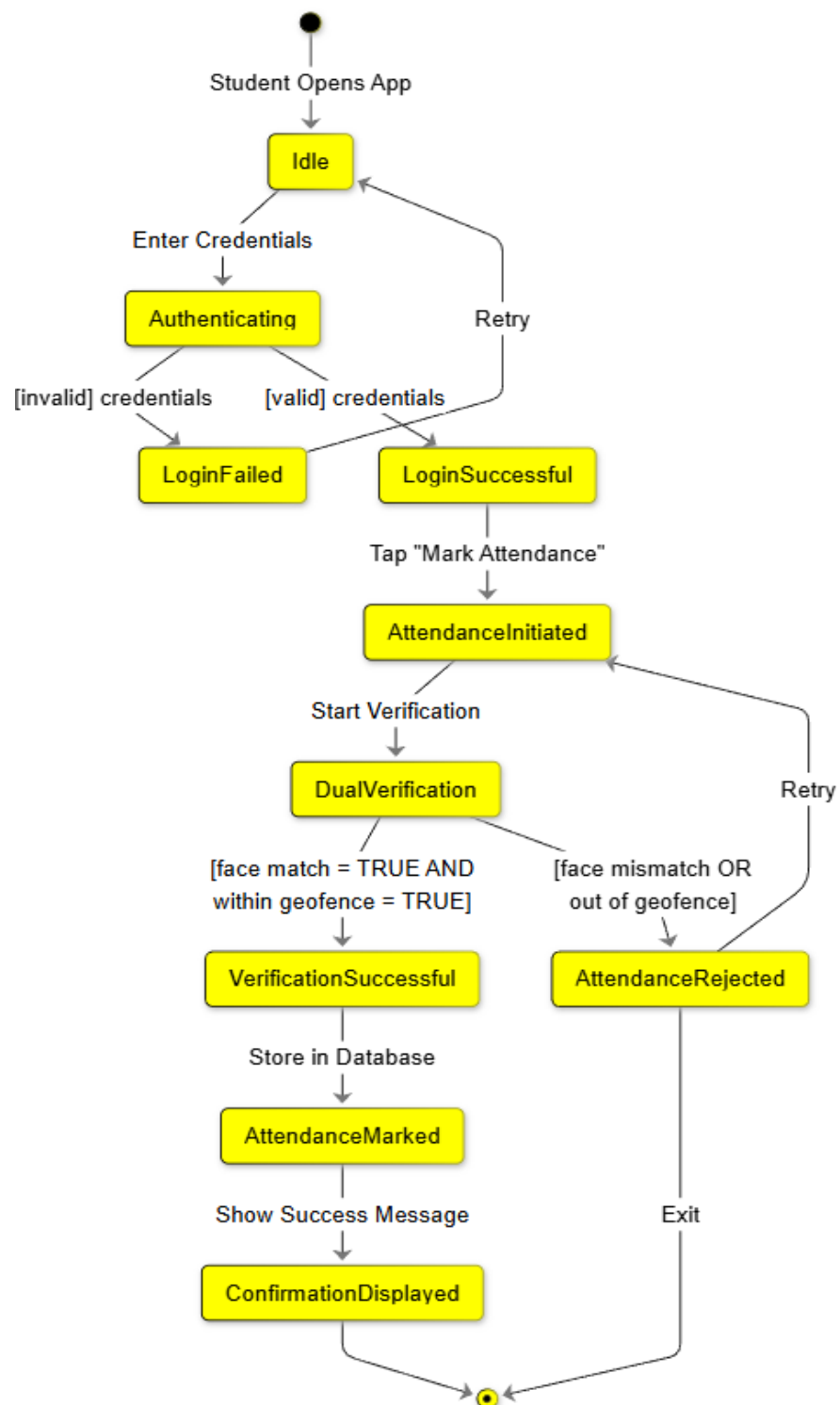


#### Case 2: Teacher Views Attendance

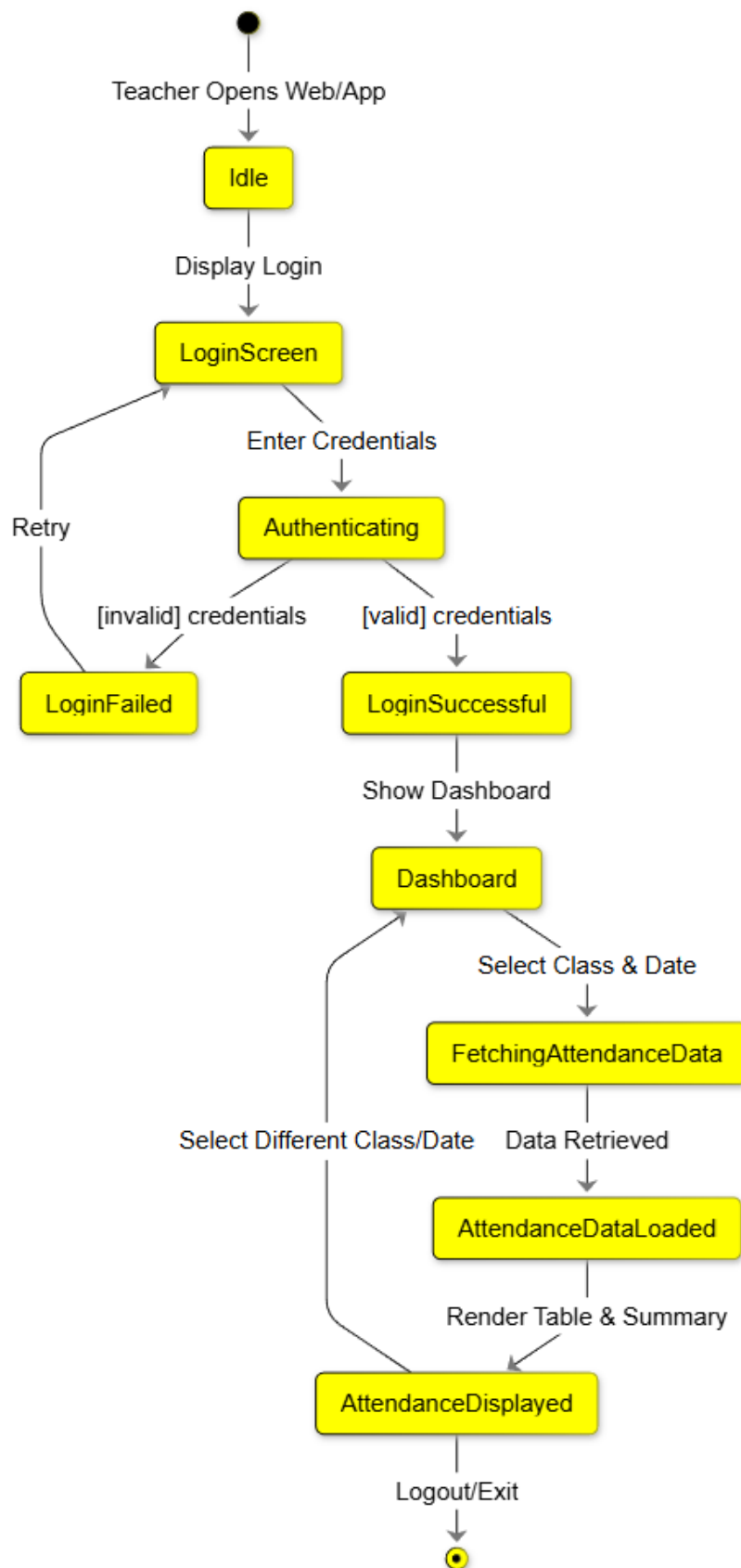


### 3.4 State Chart Diagrams

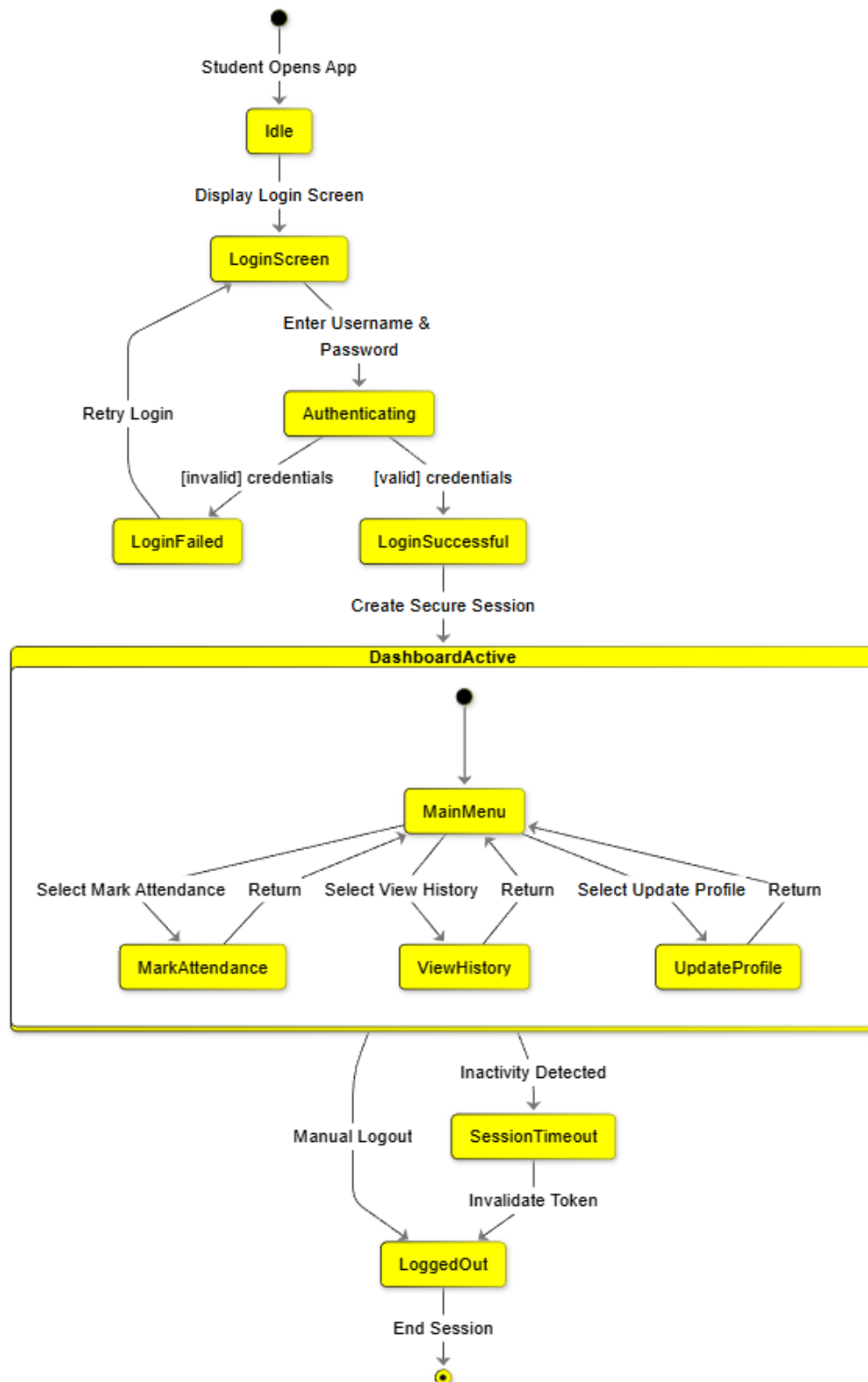
#### Case 1: Student State Chart – Attendance Process



## Case 2: Teacher State Chart – Attendance Viewing Process

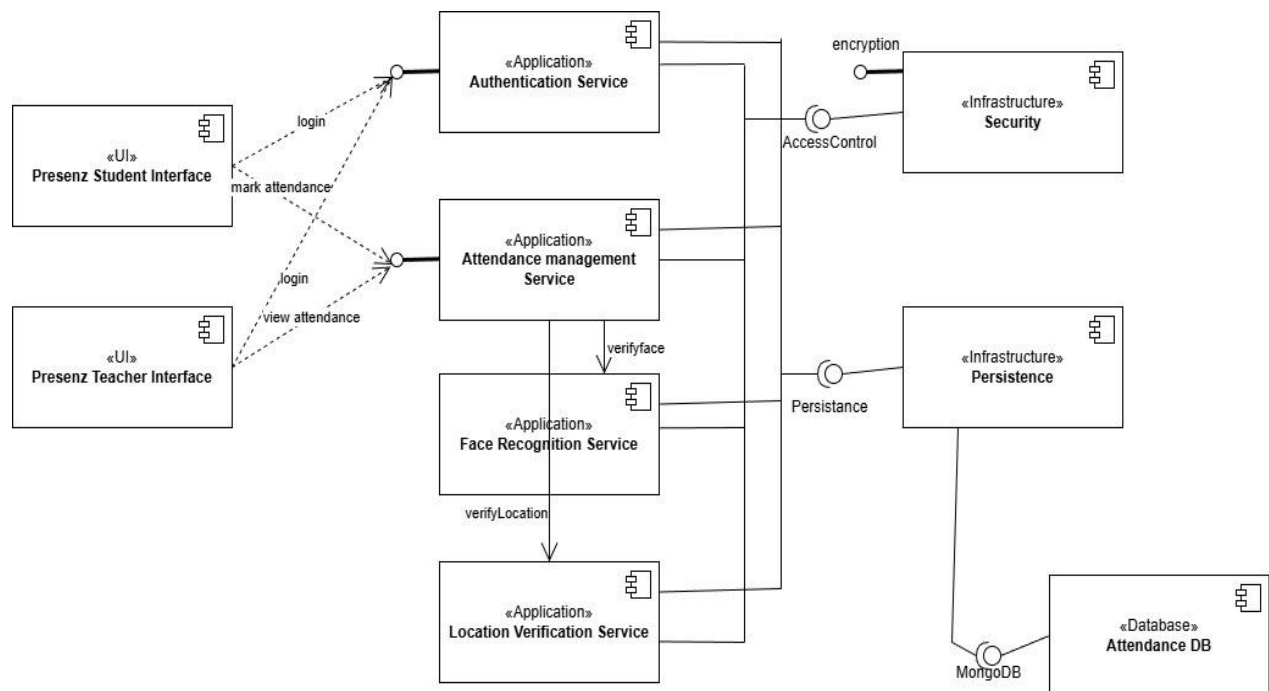


### Case 3: System / Session State Chart

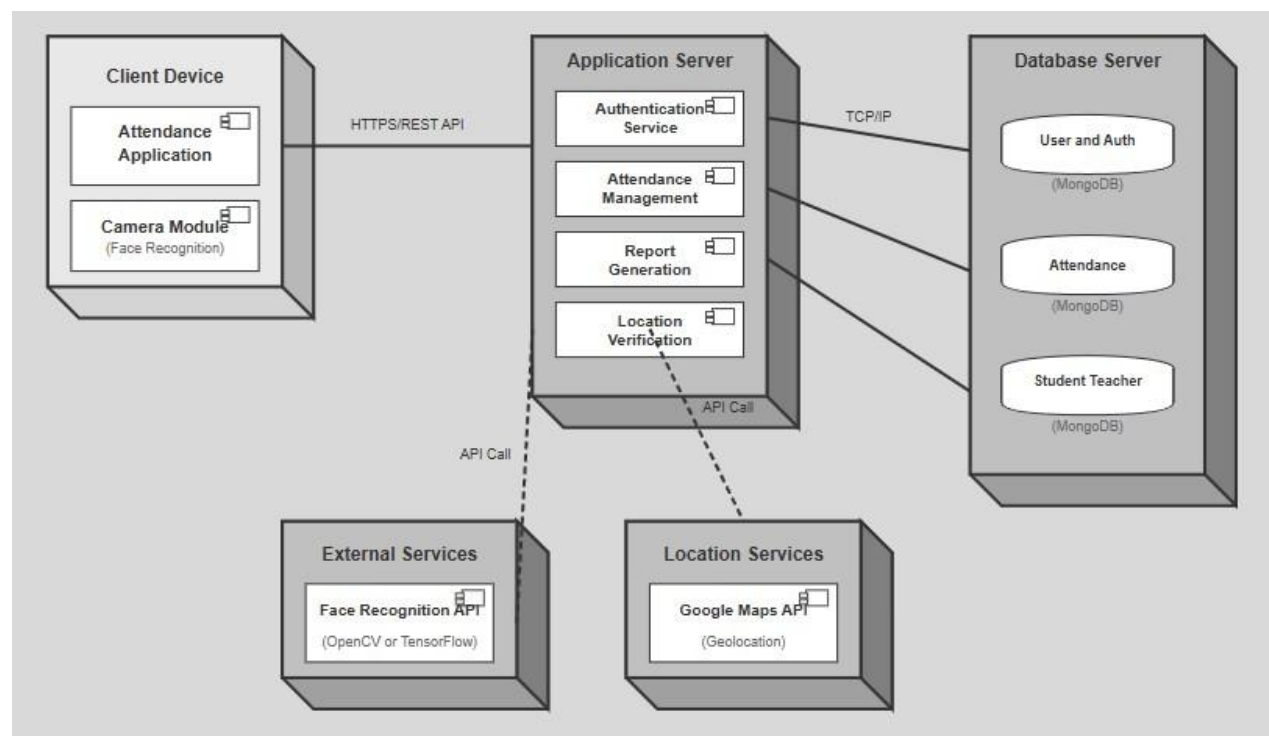


## 4. Implementation

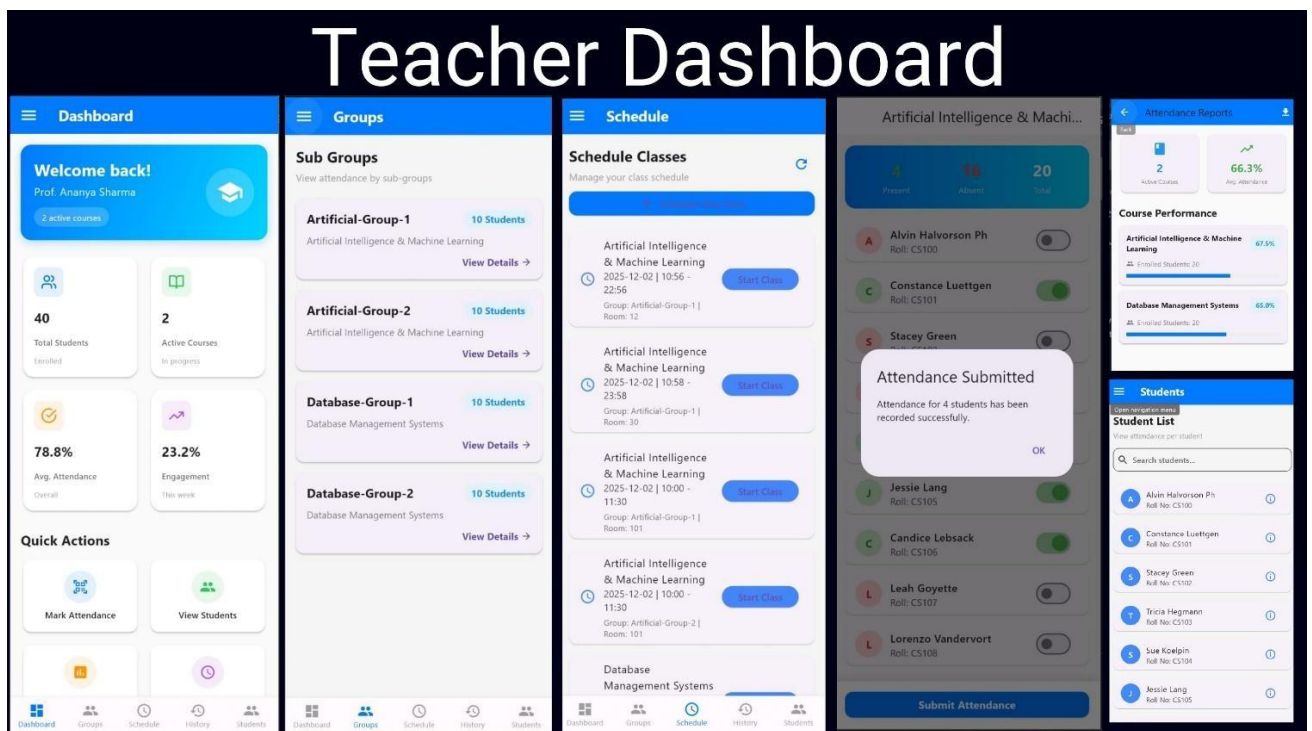
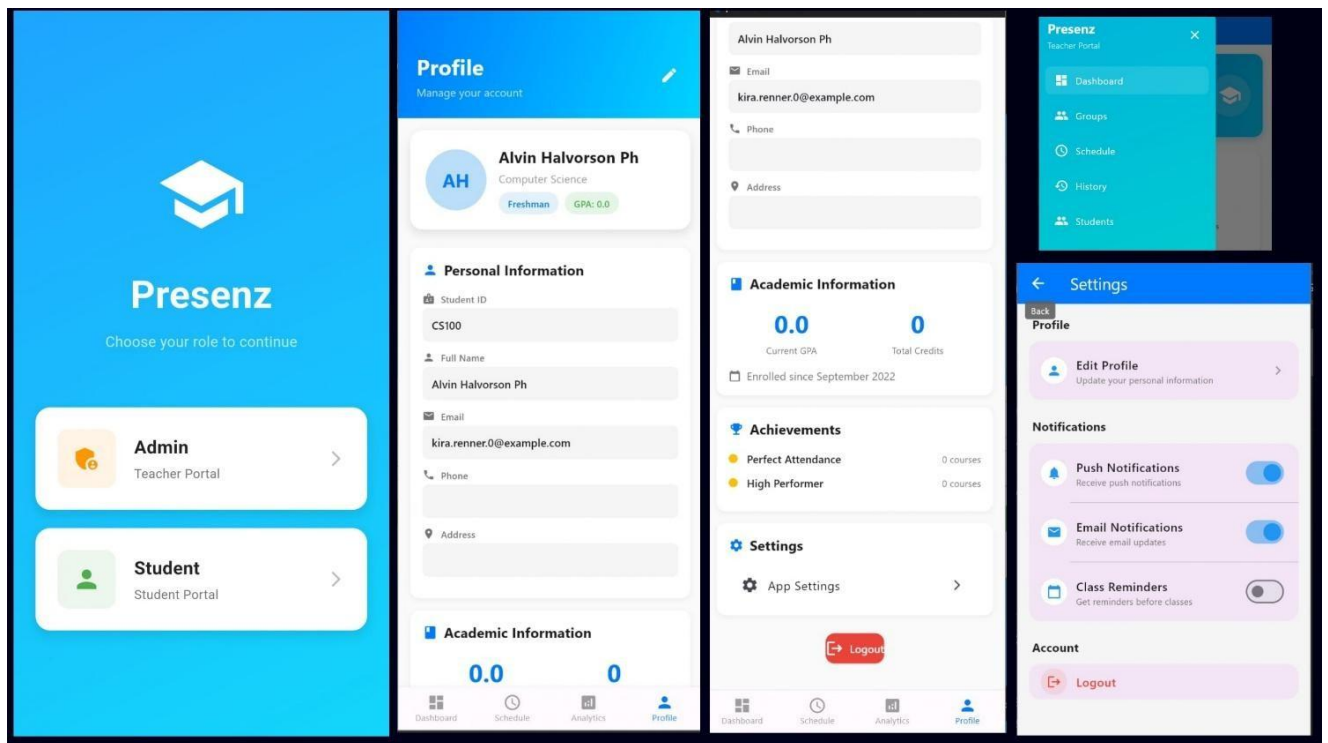
### 4.1 Component Diagram



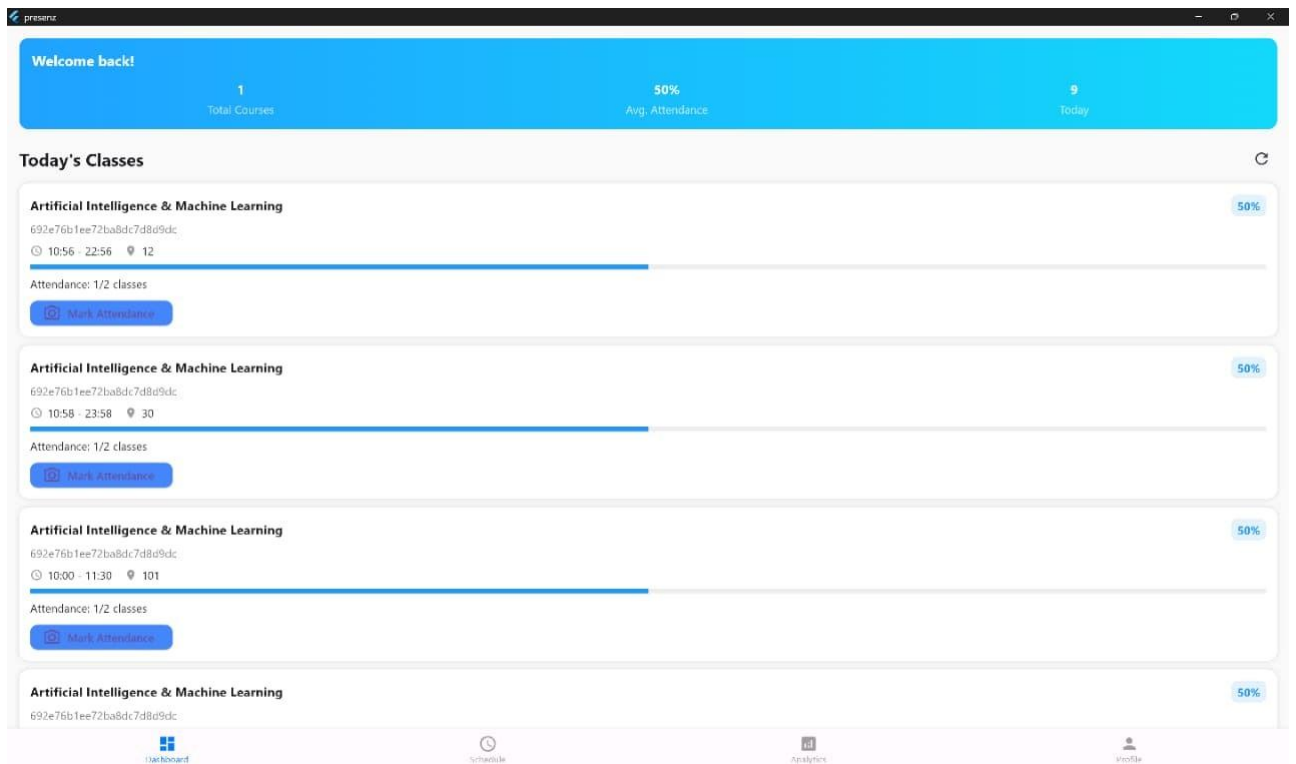
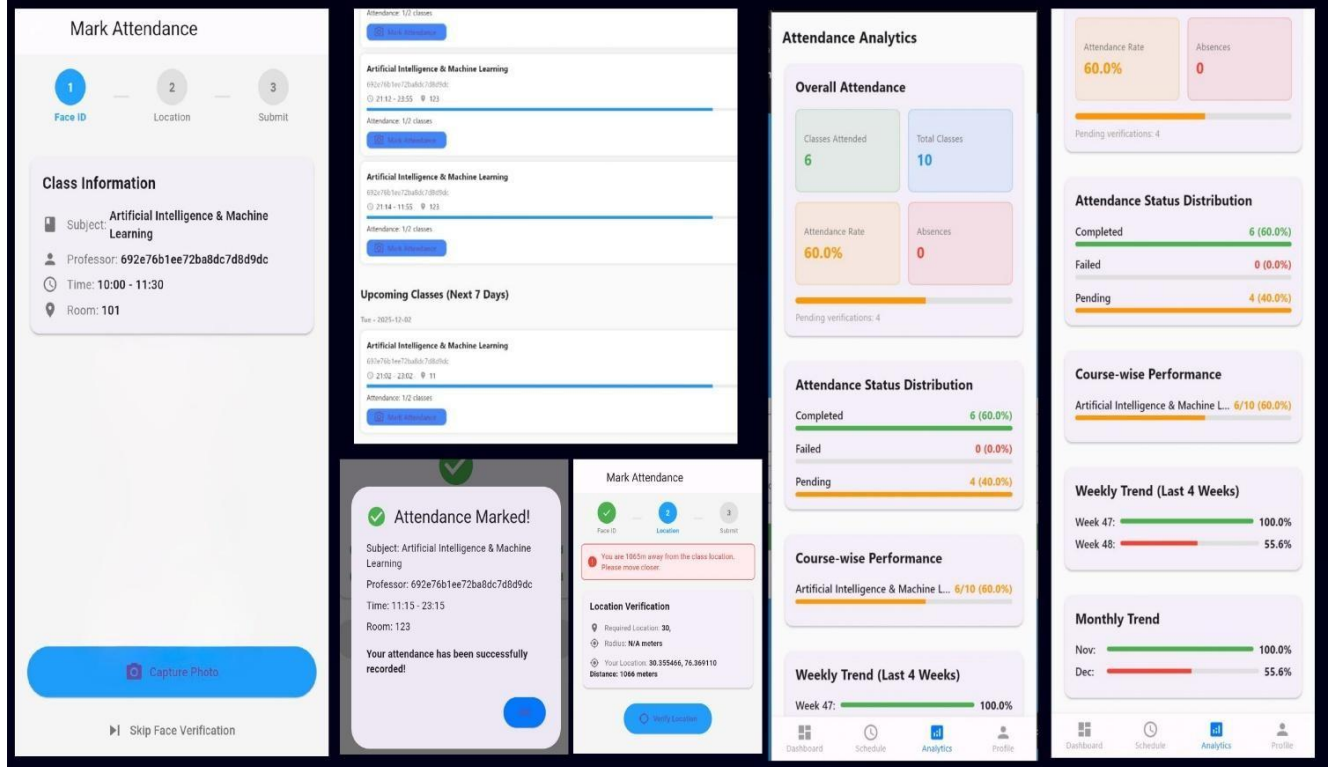
### 4.2 Deployment Diagram



## 4.3 Screenshots



# Student Dashboard



## **5. Testing**

### **5.1 Test Plan**

#### **Objectives of the Test Plan:**

- To verify that all functional requirements of PRESENZ are correctly implemented
- To ensure accurate face recognition and GPS-based location verification
- To validate secure authentication for students and teachers
- To confirm proper storage and retrieval of attendance data from the database
- To ensure the system is free from critical errors and failures

#### **Scope of Testing:**

- Student Login Module
- Teacher Login Module
- Face Recognition Module
- GPS and Location Verification Module
- Attendance Management Module
- Database Storage and Retrieval
- Security and Access Control

#### **Testing Approach:**

- Unit Testing: Individual modules such as authentication, attendance, and face recognition are tested separately.
- Integration Testing: Interaction between user interface, backend services, and database is tested.
- System Testing: The complete system is tested in a real-time environment.
- User Acceptance Testing: Final testing is done to verify that the system meets user expectations.

**Test Environment:**

- Client Devices: Mobile Phone and Laptop
- Server: Backend deployed on Render
- Database: MongoDB Atlas
- Network: Internet using HTTPS communication

**Test Entry Criteria:**

- All modules are successfully developed
- Database connection is established
- User interfaces are functional
- Required hardware such as camera and GPS is available

**Test Exit Criteria:**

- All test cases are executed successfully
- No critical defects remain
- Attendance is correctly recorded and displayed
- Database operations are verified
- Security checks are validated

## 5.2 Test Cases

**Test Case #:** 1.1

**Test Case Name:** Student Login

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test student login using valid credentials

### Pre-conditions

The student is registered in the system  
The student has a valid User ID and Password  
Internet connection is available  
PRESENZ Student Interface is installed

| Step | Action                             | Expected System Response     | Pass/Fail | Comment                   |
|------|------------------------------------|------------------------------|-----------|---------------------------|
| 1    | Open the PRESENZ Student Interface | Login screen is displayed    | Pass      | App launched successfully |
| 2    | Enter valid User ID and Password   | System validates credentials | Pass      | Valid input accepted      |
| 3    | Click Login button                 | System grants access         | Pass      | Login successful          |
| 4    | Student dashboard loads            | Dashboard is visible         | Pass      | Dashboard visible         |

### Post-conditions

1. Student session is created successfully
2. Student is redirected to dashboard

**Test Case #:** 1.2

**Test Case Name:** Mark Attendance

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test face and GPS based attendance marking

### Pre-conditions

Student is logged into the system  
Camera and GPS are enabled  
Student is inside geofence  
Internet connection is active

| Step | Action                       | Expected System Response       | Pass/Fail | Comment             |
|------|------------------------------|--------------------------------|-----------|---------------------|
| 1    | Click Mark Attendance        | Camera and GPS activate        | Pass      | Sensors activated   |
| 2    | Capture face image           | Face verification starts       | Pass      | Face image captured |
| 3    | System captures GPS location | Location verification starts   | Pass      | GPS captured        |
| 4    | Verification passes          | Attendance marked successfully | Pass      | Attendance recorded |

### Post-conditions

Attendance record is stored in database

**Test Case #:** 1.3

**Test Case Name:** Face Verification Failure

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test attendance rejection due to face mismatch

**Pre-conditions**

Student is logged into the system  
Camera is working  
Captured face does not match database

| Step | Action                  | Expected System Response | Pass/Fail | Comment          |
|------|-------------------------|--------------------------|-----------|------------------|
| 1    | Click Mark Attendance   | Camera activates         | Pass      | Camera working   |
| 2    | Capture incorrect face  | Face verification starts | Pass      | Face detected    |
| 3    | System detects mismatch | Attendance rejected      | Pass      | Mismatch handled |

**Post-conditions**

Attendance is not stored in database

**Test Case #:** 1.4

**Test Case Name:** View Attendance

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test teacher viewing attendance records

**Pre-conditions**

Teacher is registered  
Teacher is logged into the system  
Attendance data exists  
Internet is available

| Step | Action                         | Expected System Response  | Pass/Fail | Comment           |
|------|--------------------------------|---------------------------|-----------|-------------------|
| 1    | Open PRESENZ Teacher Interface | Dashboard is displayed    | Pass      | Teacher logged in |
| 2    | Select class and date          | System fetches records    | Pass      | Data fetched      |
| 3    | Click View Attendance          | Attendance data displayed | Pass      | Records visible   |

**Post-conditions**

Teacher can view attendance successfully

**Test Case #:** 1.5

**Test Case Name:** Camera Access Failure

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test behavior when camera permission is denied

**Pre-conditions**

Student is logged into the system  
Camera permission is denied from device settings  
Internet connection is available

| Step | Action                          | Expected System Response         | Pass/Fail | Comment                  |
|------|---------------------------------|----------------------------------|-----------|--------------------------|
| 1    | Click Mark Attendance           | System tries to access camera    | Pass      | Request for camera shown |
| 2    | User denies camera permission   | System detects permission denial | Pass      | Permission blocked       |
| 3    | System stops attendance process | Error message is displayed       | Pass      | User informed correctly  |

**Post-conditions**

Attendance is not marked due to camera access denial

**Test Case #:** 1.6

**Test Case Name:** Invalid Login Attempt

**System:** PRESENZ

**Designed by:** Group 5

**Short Description:** Test student login with incorrect credentials

**Pre-conditions**

Student is registered  
Student enters incorrect User ID or Password  
Internet connection is available

| Step | Action                            | Expected System Response                    | Pass/Fail | Comment                |
|------|-----------------------------------|---------------------------------------------|-----------|------------------------|
| 1    | Open PRESENZ Student Interface    | Login screen is displayed                   | Pass      | App launched           |
| 2    | Enter invalid User ID or Password | System validates credentials                | Pass      | Invalid input detected |
| 3    | Click Login button                | Access is denied and error message is shown | Pass      | Security check passed  |

**Post-conditions**

Student session is not created

### **5.3 Test Reports by Peers**

Peer testing was carried out to understand how the PRESENZ system performs when used by actual users. For this purpose, a few classmates and peers who were not part of the development team tested the application. This helped in getting unbiased feedback on the working, usability, and overall performance of the system.

During peer testing, users performed common activities such as student login, marking attendance using face and location verification, teacher login, and viewing attendance records. They also tried different situations like entering wrong login details, marking attendance multiple times, and checking camera permission behavior. Their experience while using the system was carefully observed and noted.

Most of the feedback received from peers was positive. The system worked smoothly and gave accurate results in normal usage. A few small suggestions were given related to improving the clarity of messages shown on the screen and making the interface more user-friendly. No major functional problems or critical errors were found during testing. The small issues that were identified were corrected after review.

Overall, peer testing helped in improving the quality of the PRESENZ system and increased confidence that the application is reliable and suitable for real-world use in an academic environment.